分类号 _____ 密级 _____

UDC _____ 编号 _____

中国科学院研究生院

硕士学位论文

HotSpot 虚拟机在“龙芯”Linux 系统上的移植与性能优化

陈杰

指导教师 张福新 副研究员

中国科学院计算技术研究所

申请学位级别 工学硕士 学科专业名称 计算机系统结构

论文提交日期 2007 年 5 月 论文答辩日期 2007 年 6 月

培养单位 中国科学院计算技术研究所

学位授予单位 中国科学院研究生院

答辩委员会主席 _____

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

声　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

摘 要

在“龙芯”产业化过程中，“应用软件库”不丰富是一个重要的问题，对于用 c/c++ 等平台相关语言编写的应用程序，移植和维护都需要大量的人力物力，只要“龙芯”产品投入市场，移植和维护的能力将难以满足用户需求的增长。由于 Java 程序跨平台的特性，只要龙芯上有一个稳定，性能高的 Java 平台，将使得现有的大量 Java 应用程序直接在“龙芯”产品上运行，这样将使得“龙芯”的软件库能够成倍增长。

本文的目的就在于完善现有的 1.5 版 Sun HotSpot Java 虚拟机的移植，并分析目前其性能低下的原因，改善 Java 虚拟机的性能。基于这个目的，本文研究了 Java 虚拟机的规范、字节码解释器、JIT 编译器、垃圾回收机制以及目前广泛采用的 Java 虚拟机调试方法和性能分析方法等；解决了之前遗留下来的若干问题，比如，某些情况下多线程程序出现死锁，JIT 编译执行时偶尔出错等；找出并解决了 Java 虚拟机的若干性能问题：通过改写 CACHE 操作方式解决了 JIT 安装代码阶段时间过长的瓶颈，通过处理对非对齐地址的访问消除了解释器性能低下的瓶颈等。经过这些工作，Java 虚拟机的稳定性与性能均有了大幅提高。

为了生成更高效的代码，本文还研究了 1.6 版 Java 虚拟机的移植，以进一步的提高龙芯上 Java 平台的执行性能。

关键词：Java；HotSpot； Java 虚拟机；龙芯；解释器；JIT 编译器

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

Porting and Optimization of **the HotSpot** Java Virtual Machine on Loongson

Chen Jie (Computer Architechture)

Directed by Zhang Fuxing

Java is widely used to develop software in different environments, ranging from embedded systems to high-end server applications. In the process of loongson industrialization, lacking of application software becomes an important issue.Because loongson is incompatible with X86 architecture, most of the software must be ported to loongson architecture before use. It will take a lot of labor and money to do it. Because Java is endowed with the platform independence feature, the Java application can be executed on any platform if there is a Java virtual machine (JVM), so a stable, high performance virtual machine can be a good solution to the software problem of loongson.

In this paper, The architecture of Java virtual machine (JVM)is presented, then the performance of Sun's HotSpot JVM on loongson is analyzed, after that, the CACHE operation is improved to eliminate the bottleneck of JIT compiler and the access to unaligned memory address is eliminated to improve the Interpreter's performance,at last,the HotSpot JVM of 1.6 version is analyzed and porting to loongson.Due to these work in this paper,the Java environment is improved greatly ,most of the Java application can excuete well on loongson util now.

Keywords: Java, JVM, loongson, HotSpot, Interpreter, JIT, JVM98

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

摘　要 I

第一章 引言 1

1.1 课题背景 1
1.2 Java的国内外研究现状 1
1.3 本文的工作 2
1.4 本文的组织 3

第二章HotSpot Java虚拟机的体系结构 5

2.1 类装载器 6
2.2 运行时数据区 6
2.3 执行引擎 8

第三章 HotSpot Java虚拟机 1.5 版的调试和性能优化 11

3.1 Java虚拟机的调试 11
3.1.1 调试的工具以及方法 11
3.1.2 几个重要bug的调试 13
3.2 Java虚拟机的性能分析和优化 15
3.2.1 工作集和性能测试工具 15
3.2.2 编译工具带来的优化 16
3.2.3 优化JIT编译器的瓶颈 19
3.2.4 解释器的优化 23
3.2.5 其它优化措施 27
3.2.6 本节总结 28

第四章 HotSpot Java 虚拟机 1.6 版分析与移植 33

4.1 移植到新版本Java虚拟机的必要性 33
4.2 JDK1.6 在龙芯上的移植分析 34
4.3 汇编器的移植 35
4.3.1 代码的存储 35
4.3.2 重定位 36
4.3.3 寻址方式 37
4.3.4 分支转移语句 37
4.3.5 指令生成 37
4.4 解释器的移植 37
4.5 JIT编译器的移植 38
4.5.1 线形扫描寄存器分配算法的优势 39

4.5.2 线形扫描寄存器分配算法实现 40
4.5.4 更新LIR 中间代码 44
4.6 运行时环境的移植 44
4.6.1 Java虚拟机的栈帧 45
4.6.2Java虚拟机的同步机制 46
4.7 本章总结 47
第五章 结论和展望 49
5.1 本文工作总结 49
5.2 下一步的工作 49
参考文献 i
致 谢 iii
作者简历 v

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

第一章 引言

1.1 课题背景

Java产生与流行是Internet发展的客观要求。Java平台现在已经被移植到从信用卡到大型机等多种场合，应用领域也遍及简单的网络插件到大规模的商业程序。Java平台由Java API和Java虚拟机 2 部分构成，其中Java虚拟机是Java平台的核心，它负责装载Java class文件并且执行其中的字节码。Java虚拟机的稳定性、安全性和性能随着Java应用的快速发展而变得越来越重要[1]。

Java 平台已经成为软件开发的主流工具之一，Java 软件非常丰富：在企业应用方面，基于 Java 的 J2EE 在内部众多系统的整合、系统的延展性、安全性方面对比微软的 net 平台占有优势，符合 J2EE 规范的产品重出不穷，IBM 的 WebSphere 和 BEA 的 WebLogic 等基于 Java 的应用软件在目前服务器领域占了很大比例；在中间件市场，基于 Java 的实现也逐渐成为主流；在桌面应用方面，如在办公软件方面有永中 office,在软件开发方面有 Eclipse，在个人通信平台方面有 Luma QQ 等，这些几乎构成了一整套的桌面环境；在嵌入式领域，基于 Java 的 J2ME 平台逐渐成为通信设备/消费类设备的基本功能，J2ME 应用程序的开发和部署的灵活性将有效地满足日益增长的无线领域的需求。在“龙芯”产业化过程中，软件库不够丰富是一个重要的问题，对于用 C/C++等平台相关语言编写的应用程序，移植和维护都需要耗费大量的人力物力，如果基于“龙芯”的产品大量投入市场，对应用软件移植和维护的能力将难以满足用户的需求。Java 程序天生具有跨平台的特点，无论是在何种操作系统，何种体系结构上开发的 Java 软件。都可以无差别的运行在具有符合标准的 Java 虚拟机的平台上。由于 Java 程序这种跨平台的特性，只要“龙芯”上有一个稳定、高性能、符合标准的 Java 平台，将使得现有的大量 Java 应用程序可以直接在基于“龙芯”的产品上运行，从而让“龙芯”的软件库成倍地增长。

本文的目的就在于在目前移植的 Java 虚拟机的基础上，进一步增加 Java 虚拟机的稳定性和性能，使得“龙芯”机器上 Java 程序能够基本可用，让 Java 应用程序成为“龙芯”软件库的重要补充。

1.2 Java 的国内外研究现状

Java平台于 1995 年推出，当时，虚拟机采用解释执行的方式，每条指令都由虚拟机当场解释执行，执行速度非常慢。1996 年底，Sun发布了第一个JIT编译器，JIT编译器在每段代码执行前进行编译，编译的结果为本地静态代码，执行速度有了质的提高，但由于它需要实时的编译所有程序，性能仍离C/C++有较大的差距。1999 年初，Sun发布了

第一代HotSpot引擎，利用对程序运行状况采样的结果选择性的编译Java方法。这是一种动态编译的技术，在Java虚拟机的发展上有里程碑意义。利用这种技术，Java程序大大缩小了和C/C++程序在性能上的差距，甚至在理论上有可能超过C/C++[2, 7, 9]。最近的 5 年，业界对Java虚拟机进行了进一步的优化，这些优化主要集中在 3 个方面[3, 4, 5, 6]

- 研究和优化新的采样算法，因为采样是 Java 虚拟机不停进行的一个操作，而且采样的结果关系到不同的优化策略；
- 研究深度优化的方法，这主要集中在 Java 虚拟机服务器端应用上，服务器应用一般不关注程序的启动速度，而对程序正常执行的效率非常的重视；
- 研究垃圾回收算法。垃圾回收也是 Java 虚拟机运行整个过程伴随的操作，现在的 Java 虚拟机一般还是采用 stop-the-world 的方式，即进行垃圾回收操作时，对正在运行的线程进行暂停，操作结束后恢复运行。垃圾回收的效率影响到整个 Java 虚拟机的性能。

Java应用现在非常广泛，Sun，IBM，BEA等都推出了稳定性好，性能高的虚拟机，但这些Java虚拟机均不支持MIPS体系结构。目前支持MIPS平台的Java虚拟机目前只有KAFFE[14]这个开源的项目，它几乎支持所有的Linux,UNIX和BSD操作系统，几乎支持所有的常用CPU，如X86/IA32,IA64,MIPS,ARM,PPC,Sparc,mk68k,S390 等。 虽然Kaffe对系统平台的支持非常广泛，但它是一个相当简单的Java虚拟机，仅实现了非常有限的功能，并且它使用的GNU classpath不兼容于标准的Java类库，这样会导致相当多的Java程序无法运行，比如说需要使用标准的AWT库的Java程序[14]。2005～2006 年中，中科院计算所完成了Sun的HotSpot Java虚拟机从X86 平台到MIPS平台的移植，并做了部分的优化。但由于JDK平台规模庞大，复杂性很高，加上时间仓促，移植方面有些工作需要继续完善优化，才能使基于Java软件在龙芯上达到可用的效果。

1.3 本文的工作

在前人工作的基础上，本文完善了 Sun HotSpot Java 虚拟机的移植，改善了运行的稳定性，并分析了性能瓶颈，提高了性能。具体的工作包括下面几个方面：

- 研究了 Java 虚拟机相关的各种技术，为完善 Java 虚拟机的移植与优化打下基础。Java 虚拟机是一个庞大而复杂的软件，到现在为止，已经被 Sun 公司开发了 12 年。它涉及到非常广泛的计算机理论和技术，包括 Java 虚拟机规范，字节码解释器，JIT 编译器，垃圾收集等。
- 论述调试 Java 虚拟机的方法，并叙述了几个关键的调试例子。通过调试，基本解决了 Java 虚拟机稳定性问题；
- 分析 Java 虚拟机目前在龙芯上运行的性能瓶颈，并作性能优化。主要使用了这几种方法：利用基准的 JVM98 测试分析并改进性能，对比性能；利用虚拟机自带的性能统计工具分析并改进性能；利用 Linux 的 Oprofile 工具分析并改进性

能；尝试换用不同的编译工具，编译选项来查看对性能的影响。并在这个基础上得出了龙芯上 Java 虚拟机性能不足的原因与解决方案；

- 为了解决 JIT 编译器生成代码效率低下的问题，开始了对 1.6 版本 Java 虚拟机的移植。

1.4 本文的组织

下面介绍本文的组织：

第二章介绍了 Java 虚拟机的体系结构，把 Java 虚拟机分成了类装载器，运行时数据区，执行引擎 3 部分分别进行了分析；

第三章介绍了优化 Java 虚拟机 1.5 版过程中的改善稳定性和改善性能的方法，改善稳定性方面主要是利用 GDB,JDB 调试，使用虚拟机内部运行选项调试，利用错误报表调试等，然后再论述了调试过程中遇到的几个典型问题，改善性能主要是介绍了 Java 虚拟机的性能分析和优化的方法，这部分主要是分析了 Java 虚拟机运行的性能瓶颈，以及解决这些瓶颈采用的措施，这一章是本文的重点；

第四章介绍了 1.6 版 Java 虚拟机的移植，这一章主要论述了移植 1.6 版 Java 虚拟机的一些典型问题；

第五章主要是总结和展望，对论文工作做了简单的总结。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

第二章 HotSpot Java 虚拟机的体系结构

Java 体系结构由Java程序设计语言，Java class文件格式，Java应用编程接口，Java虚拟机等 4 个部分组成。Java程序设计语言用来编写Java源代码，在这个过程调用Java应用编程接口来访问系统资源，Java编译器把它编译成Java class文件格式，即Java程序，然后再在Java虚拟机中运行class文件。Java虚拟机和Java应用编程接口（Java API）构成了Java平台。Java程序的跨平台特性，主要是由于Java平台屏蔽了平台相关部分。Java虚拟机是运行Java程序的抽象计算机，它是Java体系结构的核心。其规范定义了每个Java虚拟机都要实现的特性，但具体的实现方式是灵活多样的，甚至某些特性可以利用硬件来实现，Java规范的灵活性保证了它能够在广泛的计算机或设备上得到实现[8]。

Java虚拟机的体系结构如图 2-1 所示[9]。

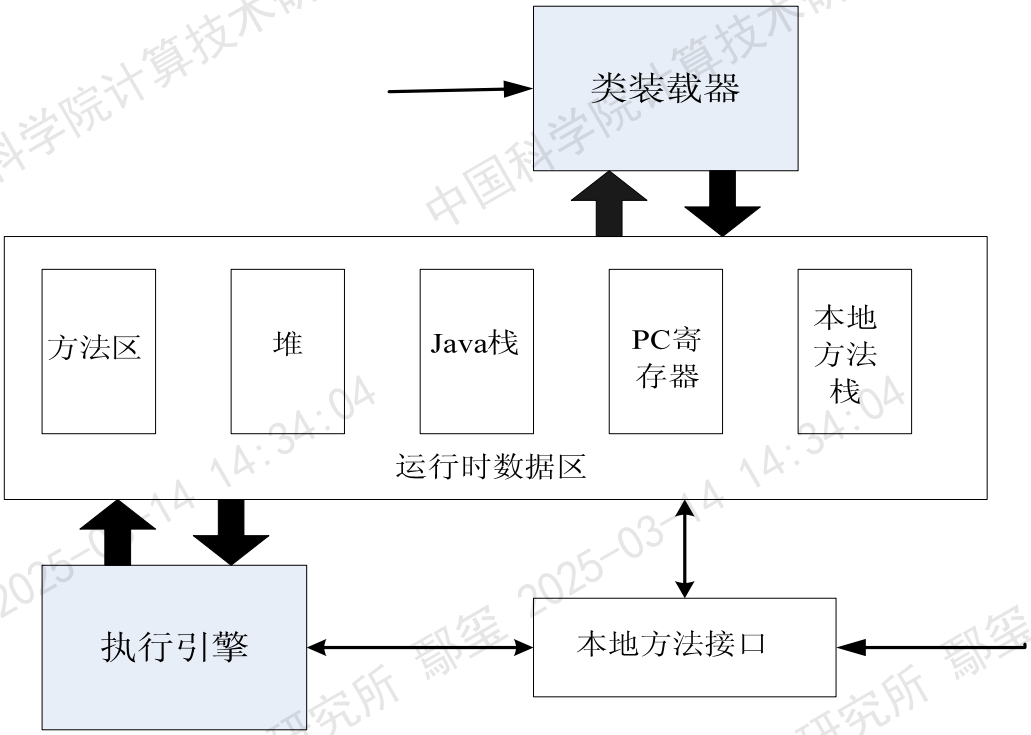


图 2-1 Java 虚拟机体系结构图

当一个 Java 程序开始运行时，它就创建了一个 Java 虚拟机实例。每个 Java 虚拟机实例都有一个类装载子系统，它根据给定的全限定名来装入类型，同样，每个 Java 虚拟机实例都有一个执行引擎， 它负责执行那些包含在被装载类的方法中的指令。当 Java 虚拟机运行一个程序的时，它需要“运行时数据区”来存储字节码，class 文件中的相关信息，程序创建的对象，传递给被调用方法的参数，方法返回值，局部变量，以及运算的中间结果等。其中某些数据区是由程序中所有线程共享的，还有一些则是线程独有的，每个 Java 虚拟机实例都只有一个方法区及一个堆，它们由所有线程共享的。当虚拟机装

载一个 class 文件时，它会从这个 class 文件包含的 2 进制数据中解析类型信息。然后把这些类型信息放到方法区中。当程序运行时，虚拟机会把所有该程序运行时创建对象都放到堆中，每当一个新的线程被创建时，它都将得到它自己的 pc 寄存器，以及一个 Java 栈，如果线程正在执行的是一个 Java 方法，那么 pc 寄存器的值总是指示下一条将被执行的指令。而它的 Java 栈则总是存储该线程中 Java 方法调用的状态，包括它的局部变量，传进来的参数，返回值，以及运算中的中间结果等等。而本地方法调用的状态，则是以某种依赖于具体实现的方式存储在本地方法栈中，Java 栈是由许多的栈帧组成，一个栈帧包含一个 Java 方法调用的状态，当线程调用一个 Java 方法时，虚拟机压入一个新的栈帧到该线程的 Java 栈中，方法返回时，这个栈帧被从 Java 栈中弹出并抛弃。

2.1 类装载器

Java 虚拟机有 2 种类装载器：启动装载器和用户自定义类装载器。前者是虚拟机实现的一部分。后者是 Java 程序的一部分。由不同的类装载器装载的类被放到虚拟机内部的不同的命名空间。对于每一个被装载的类型，Java 虚拟机都会为它创建一个 Java.lang.class 类的实例来代表该类型。和所有其它对象一样，用户自定义的类装载器以及 class 类的实例都放在内存中的堆区，而装载的类型都位于方法区。装载，链接以及初始化，类装载器子系统除了要定位和装入 class 文件外，还负责验证被导入类的正确性，为类变量分配并初始化内存，以及帮助解析符号引用。这些动作必须严格按照如下顺序进行：

- 装载：查找并装载类型的二进制数据
- 连接：执行验证，准备，以及解析
- 验证：确保被导入类型的正确性
- 准备：为类变量初始化内存，并将其初始化为默认值
- 解析：将类型中的符号引用转换为直接引用
- 初始化：把类变量初始化为正确的初始值

2.2 运行时数据区

方法区，堆区，PC 寄存器，栈区，本地方法栈构成了运行时数据区，本节将分述这 3 个部分。

- 方法区

当虚拟机装载某个类型时，它使用类装载器定位相应的 class 文件，然后读入这个 class 文件，接着 Java 虚拟机提取其中的类型信息，并将这些信息存储到方法区，该类型中的类(静态)变量同样也是存储在方法区中。对每个被装载的类型，虚拟机都会在方法区存储如下类型的信息：这个类型的全限定名，这个类型直接超类的全限定名，这个类型是普通类还是接口类，这个类型的访问修饰符，任何直接超接口的全限定名的有序列

表。除了上面的基本类型信息，虚拟机还得为每个被装载的类型存储以下信息：该类型的常量池，字段信息，方法信息，除了常量以外的所有类变量，一个到 classloader 的引用，一个到 class 类的引用。由于所有的线程都共享方法区，因此位于方法区的数据的访问必须被设计为线程安全的。比如，假设同时有 2 个线程都企图访问一个名为 Lava 的类，而这个类还没有装入虚拟机，那么，这时只应该有一个线程去装载它，而另一个线程只是等待。方法区的大小不必是固定的，虚拟机可以根据应用的需要动态调整。同样，方法区也不必是连续的，方法区可以在堆中自由的分配。方法区也可以被垃圾回收，因为虚拟机允许通过用户定义的类装载器来动态扩展 Java 程序，因此一些类也会成为程序不再引用的类，当某个类变为不再引用的类时，Java 虚拟机可以卸载这个类， 从而使得方法区占据的内存保持最小。

➢ 堆

Java 程序运行时创建的所有类实例或数组都放在同一个堆中，而一个 Java 虚拟机实例中只存在一个堆空间，因此所有的线程都将共享这个堆。又由于一个 Java 程序独占一个 Java 虚拟机实例，因而每个 Java 程序都有自己的堆空间，即它们不会彼此干扰。但同一个 Java 程序的多个线程共享着一个堆空间，在这种情况下，就得考虑多线程访问对象的同步问题了。和方法区一样，Java 堆也不必是连续的内存区。

➢ PC 寄存器

对于一个运行中的 Java 程序而言，其中的每一个线程都由它自己的 PC 寄存器，它是在线程启动时创建的，PC 寄存器的大小是一个字长，因此它能够维持一个本地指针，也能够持有一个返回地址，当线程执行某个 Java 方法时，PC 寄存器的内容总是下一条被执行指令的地址，这里的地址可以是一个本地指针，也可以是在方法字节码中相对于该方法起始指令的偏移量。如果该线程正在执行一个本地方法，那么 PC 寄存器的值是未定义的。

➢ 栈帧

栈帧由 3 部分组成：局部变量区，操作数栈和帧数据区。局部变量区和操作数栈的大小要视对应的方法而定，它们是按字长计算的，编译器在编译时就确定了这些值并放在 Java class 文件中，“帧数据区”的大小依赖于具体实现。

局部变量区：Java 栈帧的局部变量区被组织为一个以字长为单位的，从 0 开始计数的数组，即字节码指令通过从 0 开始的索引来使用其中的数据。类型为 int，float，reference 和 returnAddress 的值在数组中只占据一项，且类型为 byte，short 和 char 的值在存入时被转化为 int，但类型为 long 和 double 的值在数组中占连续的 2 项。局部变量区包括对应方法的参数和局部变量，编译器按照声明的顺序把参数放入局部变量数组中。

操作数栈： 和局部变量区一样，操作数栈也是被组织为一个以字长为单位的数组。但和前者不同的是,它不是通过索引来访问，而是通过标准的栈操作－压栈和出栈来访问的。不同于硬件，Java 虚拟机没有寄存器，程序计数器也无法被程序指令直接访问。Java 虚拟机的指令是从操作数栈中而不是从寄存器中取得操作数的，因此它的运行方式是基于

栈的而不是基于寄存器的。虽然指令也可以从其它地方取得操作数，比如从字节码流中跟随在操作码之后的字节中或从常量池中，但主要还是从操作数栈获得操作数。虚拟机把操作数栈作为它的工作区－大多数指令都要从这里弹出数据，执行运算，然后把结果压回操作数栈。

➢ 本地方法栈

一个运行中的 Java 程序，还可能用到一些跟本地方法相关的数据区。当某个线程调用一个本地方法时，本地方法不受 Java 虚拟机的限制，它可以通过本地方法接口来访问虚拟机的“运行时数据区”，任何本地方法接口都会使用某本地方法栈。当线程调用 Java 方法时，虚拟机会创建一个新的栈帧并压入 Java 栈。当调用本地方法时，虚拟机就会保持 Java 栈不变，只是简单的动态连接并直接调指定的本地方法。如果虚拟机实现本地方法接口使用的是 C 连接模型，那么它的本地方法栈就是 C 栈。当 C 程序调用一个 C 函数时，其栈操作是确定的，传递给该函数的参数以某个确定的顺序压入栈，它的返回值也是用确定的方式传回给调用者。如果本地方法接口需要回调 Java 虚拟机中的 Java 方法，该线程会保存本地方法栈的状态进入另一个 Java 栈。就像其它“运行时数据区”一样，本地方法占用的内存区不必是固定大小的，它可以根据需要动态扩展或者收缩。

2.3 执行引擎

任何Java虚拟机实现的核心都是它的执行引擎。在Java虚拟机的规范中，执行引擎的行为使用指令集来定义, 对于每条指令，规范都详细地规定了应该完成什么样的功能[8]。

Java class 文件中的方法是由 Java 虚拟机的指令序列构成的，每一条指令包含一个单字节操作码，后面跟 0 个或者多个操作数， 操作码表明需要执行的操作，操作数指向 Java 虚拟机执行操作码时需要的额外信息。操作码本身已经规定了它是否需要跟随操作数，以及如果有操作数的话，它是什么形式的。根据操作码的需要，除了跟随操作码的操作数外，可能还需要从另外的存储区得到操作数，比如说，当虚拟机执行一条指令的时候，可能使用当前常量池中的项，当前桢的局部变量中的值，或者位于当前帧操作数顶端的值。执行引擎会不时的遇到请求本地方法调用的指令，在这个时候， 虚拟机尝试调用这个本地方法，如果本地返回了，执行引擎继续执行下一条指令。

Java 虚拟机指令集关注的中心是操作数栈。虽然 Java 虚拟机没有实际的寄存器，但每个方法都有一个局部变量集合。指令集实际的工作方式就是把局部变量当作寄存器，用索引来访问。 在使用保存在局部变量中的值之前，必须先将它压入操作数栈。平台无关性是影响指令集设计的最大因素。指令集这种以栈为中心，而非以寄存器为中心的设计方法，使得在那些只有很少寄存器，或者寄存器很没有规律的机器上实现 Java 更加便利，X86 就是一个例子，由于指令集具有这种以栈为中心的特征，所以在很多平台的体系结构上都很容易实现 Java 虚拟机。

Java 虚拟机规范对指令的执行言之甚少。Java 虚拟机的设计者可以自由决定如何执行字节码：解释执行，即时编译执行，硬件协处理器执行，或者是它们的混合，只要它们遵虚拟机指令集的定义。最有意义的执行技术之一是自适应编译执行。Sun 的 HotSpot 虚拟机就采用了这种方法。自适应编译执行器收集那些只在运行时才有效的信息，试图以某种方式把“解释执行”和编译成本地代码后执行结合起来，以得到最优化的性能。采用自适应编译器的 Java 虚拟机开始的时候对所有的代码都解释运行，但在程序运行时会监视代码的执行情况，所以虚拟机可以找出到程序的热区，即花费 80%~90%的时间来执行的那 10%~20%的代码。当自适应的虚拟机判断出某个特定的方法是瓶颈的时候，它启动一个后台线程，把字节码编译成本地代码。和传统的 JIT 编译器相比，自适应编译器有诸多的优点：

- 因为只需要编译热点函数，虚拟机可以比传统的 JIT 编译更注重性能。自适应优化技术使程序最终能把原来占 80%~90%运行时间的代码变为极度优化的本地代码，而编译使用的内存量并不比全部解释 Java 程序大多少。自适应优化的虚拟机可以保留以前的字节码，等待方法从热区移出。方法变得不再是热区时，取消那些编译过的代码，重新开始解释执行那些字节码。
- 设计良好的 Java 程序的一个特点是方法调用和动态派发高频度发生，它们从 2 个方面影响性能。首先，每次动态派发都会产生相关的管理费用；其次，更重要的是方法调用降低了编译器优化的效性。频繁的方法调用会使的编译优化的有效性降低， 因为编译优化在不同的方法调用之间不能有效地工作。方法的调用频度越高，方法调用之间可以用来优化的代码就越少， 通过“方法内嵌”可以消除方法调用，因而可以让编译优化处理更多的代码， 但代价就是需要更多的运行时内存。动态派发意味着一个函数调用可能有多个函数实现，自适应编译在运行时工作，它可以有效地根据统计的信息去选择合适的方法内嵌，能够有效的减少优化后的代码大小。

运行中的每个线程都是一个独立的虚拟机执行引擎的实例。从线程生命周期的开始到结束，它要么在执行字节码，要么在执行本地代码。一个线程可能通过解释或者使用芯片级的指令直接执行字节码，或者间接通过即时编译器执行编译过的本地代码。Java 虚拟机的实现可能用一些对用户看不见的线程，比如垃圾收集器。这样的线程不是实现的执行引擎的实例。

Java虚拟机中的线程一般使用本地线程实现，与线程相关的 2 个概念就是线程优先级和线程之间的同步。Java虚拟机一般把Java线程的优先级映射到本地线程的优先级上。Java虚拟机规范对于不同级别的线程行为，只规定了所有最高级别的线程会得到大多数CPU时间。级别低的线程只有在所有比他优先级更高的线程全部阻塞的情况下才能保证得到CPU时间，级别低的线程在级别高的线程没有阻塞的时候也可能得到CPU时间。要协调多线程之间的活动，应该使用同步：任何Java虚拟机线程实现都必须支持同步的 2 个方面：对象锁定，线程等待和通知。对象锁定使独立运行的数据访问共享数据时互斥。

线程等待和通知使得线程为了达到同一个目标而互相协同工作。运行中的程序通过Java虚拟机指令集来访问上锁机制。还通过object类的wait, notify , notifyAll方法访问线程等待和通知机制[9]。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

第三章 HotSpot Java 虚拟机 1.5 版的调试和性能优化

在本文之前，Java 虚拟机 1..5 版的移植工作已经基本完成。剩下的主要工作任务是解决不稳定问题和性能低下问题。本章首先描述在完善移植工作过程中，采用的主要调试方法以及使用这些方法解决的几个重要 bug，然后对 Java 虚拟机的性能做了分析和优化。

3.1 Java 虚拟机的调试

Java 虚拟机对于本地机器而言是一个普通的应用程序，但对于 Java 程序而言是一台虚拟的计算机，所以，对 Java 虚拟机的调试不同于对普通本地应用程序的调试，也不同于对 Java 程序的调试。在调试中必须借助多种手段来完成错误的定位，下面分节来描述工作中用到的几种调试工具或方法，再举例说明工作中几个重要 bug 的调试过程。

3.1.1 调试的工具以及方法

- 使用 GDB，JDB 来调试 Java 虚拟机

Java 虚拟机对于操作系统而言是一个应用程序，GDB 是 linux 系统下最主要的，开源的，强大的调试工具。利用 GDB 来调试 Java 程序也是调试中使用的主要手段，Java 虚拟机的核心是 libjvm 库，在 Linux 底下，对应的是 libjvm.so。使用 GDB 主要是对这个动态库进行调试。

生成 GDB 需要的带调试符号信息的文件需要在编译时加-g 选项，由于 Java 虚拟机的庞大以及工程的复杂性，在生成调试版本时还遇到了不少难题。用 GCC4.1 编译加上-g 参数编译出可调试版本后，发现 Java 虚拟机在启动时总是报错 segment fault，导致程序崩溃，无法展开调试。用如下的方法找出了这个错误的根源：

1) 尝试用 strace 跟踪程序的运行，没有发现什么异常，使用 dmesg 可以得到出错时执行指令的虚拟地址和出错时访问的地址，发现出错时指令位于 libjvm.so 的地址空间，但程序访问的地址位于系统空间；
2) 用 ld 得到 libjvm.so 在该进程中的首地址，这个首地址和出错时指令的地址相减，得到一个偏移值，在 “libjvm.so+偏移值”这个位置可以找出这条具体的指令，通过反汇编的结果，得知这是一个普通的函数调用，对应到源码，被调用的目标函数仍然位于 libjvm.so 中，这个地址显然不应该位于系统空间；
3) 找到目标函数在 libjvm.so 的地址，加上 libjvm.so 在进程中的首地址，可以得到目标函数真实的虚拟地址，这个地址与错误地址的差值正好等于 libjvm.so 的首地址。这说明函数调用时，函数地址没有加入偏移，反汇编 libjvm.so 的 got 表（全局偏移表），发现 got 表中这个首地址偏移全部没有加上，这说明 GCC4.1

编译器没有正确的编译出库文件；

4) 换用 GCC3.4 编译，发现生成的 object 文件，体积比对应的 product 版本增大了 100 倍，比 GCC4.1 编译的调试版本增大了 10 倍多，且由于本身 Java 虚拟机工程庞大，最后对象文件加起来大小超过 3g，这样导致在普通得 pc 上没有足够的存储空间来完成链接，使用 readelf 显示 object 文件的 section 有 20000 左右，其中很多都是没有意义的 section 这也许是 mips 版本 GCC3.4 的一个 bug；
5) 由于使用 GCC4.1 编译的 product 版本没有出现 got 表不加偏移的现象，经过对照 GCC4.1 编译 Java 虚拟机的调试版本和非调试版本的参数，经过尝试，得知加上-O 的优化选项后，-g 编译 Java 虚拟机才能得到可以正常运行的库，这也许是 MIPS 版本 GCC4.1 的一个 bug。这样处理的带来了一些弊端，-O 选项会对代码作一些优化，给调试带来困难。

编译出 Java 虚拟机正确的可调试 libjvm.so 之后，调试过程中还需要注意 2 点；

1) 由于 libjvm.so 这个文件不位于系统常用的库的目录中，在使用 GDB 调试 Java 前，要给 ld 设置一个 LD_PRELOAD 的环境变量。
2) 由于 Java 虚拟机定义一些自定义的信号处理函数，GDB 需要忽略对这些信号的处理，比如说 SIGSEGV，不然 Java 虚拟机无法正常执行。

但目前“龙芯”平台上 GDB 还存在缺陷：一是 watch 功能目前无法使用，二是 break 在复杂的多线程环境下无法正常停止程序的执行。为了克服后者，在源码上需要 break 的地方加上了等待输入某特定字符的语句，来模仿 break，continue 的功能。

GDB 的通用的使用方法不再详述。

JDB 是 Sun JDK 自带的一种调试工具，主要用来对 Java 程序进行调试。Java 程序对应的本地代码由 Java 虚拟机动态生成，没有任何的符号信息，调试信息等，这些部分的代码，在 GDB 的调用堆栈中表现为????，所以 GDB 的调时还需要 JDB 作为补充，用来定位错误发生时，正在执行什么样的 Java 程序。

GDB 和 JDB 结合起来使用，基本上可以定位出错时函数调用情况。

➢ 利用 Java 虚拟机崩溃时生成的错误报表

虚拟机执行过程出现崩溃时，会生成形如“hs_<errorpid>*.log” 的错误报表，这些错误报表详细记录了崩溃时的执行环境信息，便于调试。我们可以从错误报表中得到如下的信息[7]：

1) 虚拟机崩溃的原因，比如说如果是信号没有得到处理，会报出什么信号导致了崩溃，这个信号是 CPU 在执行什么地址的指令时发出的，该指令所在的动态库以及地址，当前的进程号，线程标识号等；
2) 当前的寄存器值，由寄存器的值，可以推测当前 SP 的值，当前返回地址，从而可以得到更详细的函数调用情况，当前部分栈顶的值，当前执行的指令；
3) 当前的 Java 虚拟机堆栈里面帧调用情况，包括解释帧，编译帧，Java 虚拟机帧等；

4) 当前 Java 虚拟机运行存在的线程，包括 Java 线程和本地线程等，并指出了当前线程；
5) 当前 Java 虚拟机执行的状态，比如说是普通的执行还是位于 Safepoint 等，当前同步信号量情况等；
6) 堆的地址范围分配，利用率等，这一部分主要说明垃圾回收的情况；
7) 当前 Java 虚拟机进程空间的地址分配，包括哪些动态库占用了哪些地址等；
8) 当前 Java 的执行环境，执行参数，Java 虚拟机版本信息等；

通过研究错误报表，可以对 Java 虚拟机的崩溃的原因有个大致的了解。在工作过程中，Java 虚拟机的 bug，大部分是反映在 Java 虚拟机崩溃这个现象上，所以错误报表对调试起到了比较重要的作用。

但要做到正确地利用错误报表，需要意识到报表反映的错误很有可能不是错误的根源，要让错误报表所报错误尽可能接近错误的根源，应该使用 Java 虚拟机调试版本产生的错误报表，因为 Java 虚拟机的调试版本比普通版本多了很多验证 assert 语句，这些 assert 语句可以尽可能早地检测到错误。

- 利用 Java 虚拟机内部的调试选项报表

Java 虚拟机本身带有很多调试选项，在 HotSpot/runtime，HotSpot/c1，HotSpot/cpu/gs2 目录下均有*globals.hpp 等文件，这些文件里面定义了虚拟机启动的选项，但其中很多选项在 Java 虚拟机的调试版本中才可以使用。这些选项在启动 Java 虚拟机时加“-XX：”使用，比如：Java -XX:+PrintLIRWithAssembly HelloWorld。工作中使用得比较多的选项有下面几条：

- TraceClassLoading：在一个 class 被载入时，打印出这个 class 的基本信息
- PrintSafepointStatistics：打印出垃圾回收触发的 Safepoint 的统计
- TraceBytecodes：跟踪字节码的执行，一般用来调试解释器
- PrintCompilation：打印出被编译的方法的基本信息
- CIPrintMethodCodes：打印出被编译方法的字节码
- PrintIR：打印出被编译方法 IR 中间代码
- PrintLIR：打印出被编译方法 LIR 中间代码
- PrintAssembly：打印出被编译方法对应的本地代码
- PrintIRWithLIR：打印出被编译方法 IR 代码和对应的 LIR 代码
- PrintLIRWithAssembly：打印出被编译方法 LIR 中间代码对应的本地代码
- 灵活的利用 Java 虚拟机自带的调试选项以及选项的组合，可以得到很多错误发生的详细信息。

3.1.2 几个重要 bug 的调试

本节主要论述使用前节所述的方法来调试 Java 虚拟机的几个实例。

(1) 多线程死锁 bug 的调试

这个 bug 表现为一些多线程的应用程序执行不正常，比如说在一些小游戏中键盘没响应，在一些 applet 程序中装载 class 文件时程序锁住等。

Java 虚拟机运行时加-Xint 参数，使之解释执行，没有发现错误的发生，那说明错误只有在编译执行时才会出现。知道了这些特征，基本上可以写出一个小的 demo 程序来反映出这种错误，由于编译执行才触发错误，在 demo 小程序中需要编译执行的函数内加上一个 while 超过 1500 次的循环，利用 HotSpot Java 虚拟机编译执行的机制触发该函数的编译。

当 demo 程序完成后，使用 JDB 调试，发现死锁出现在方法调用前。接下来利用-XX：+ PrintCompilation –XX：+ PrintLIRWithAssembly 选项执行应用程序，这样可以打印出哪些方法正在被编译，以及编译出来的中间代码，本地代码信息，从这些信息中可以发现，被编译的函数中有一个 Safepoint 触发点。使用-XX:+ PrintSafepointStatistics 选项执行程序，发现 Safepoint 的统计并未完成，那么现在可以确定，线程的死锁发生在处理 Safepoint 的过程中。

启动 GDB 用来调试 Java 虚拟机的 libjvm.so 动态库，在程序死锁时用 CTRL+c 停止程序的运行。通过查看程序的调用堆栈和线程信息等，发现某个线程的 pthread 线程锁的值为一个很大的正数，而这个值正常情况下只能为 0 或者 1 。此时怀疑可能是这些 GDB 中显示的线程中有一个线程破坏了堆栈。

因为虚拟机崩溃产生的错误报表中有当前进程中线程的信息，在 safeppoint 处理中死锁前添加触发虚拟机崩溃的语句，重新编译后，运行 Java 程序，得到错误报表，从错误报表中可以得出进程中存在的线程的线程号以及描述，虽然运行程序时每次创建线程的标识号不同，它们的大小顺序是一致的。

接下来再次运行 Java 虚拟机，然后仔细分析死锁处每一个线程的执行是否正确。通过查看每个线程的调用堆栈，发现有一个线程处于一个条件循环中，而这个线程正好与 Safepoint 相关，如果线程没有从这个循环中退出，那毫无疑问整个程序会陷入死锁的状况。这个线程判断当前进程又没有停止，如果没有停止则等待，然后再判断。为了确认这一点，再次修改 Java 虚拟机此处的源代码，编译运行后，证实了这个猜测。

通过研究源代码可以发现，此处 Java 虚拟机需要收到一个 SEGSEV 信号，才能停止线程，而 SEGSEV 信号由于读一个“不可读区域”触发，现在的关键是这个信号没有被触发。没有被触发的原因是 CPU 访问不可读区域地址时，使用了 ZERO 寄存器做为目的寄存器，而“龙芯-2e”出于其它方面的考虑，正好禁止了这种情况下的例外。通过改写此处的代码，使用 AT 寄存器作为目标寄存器，同时保证使用这个寄存器的安全，这个 bug 得以解决。

(2) 关于 JIT 编译器的一个 bug 的调试

在使用 Sun 公司 Java 2D demo 程序时，某些测试的显示会出现问题，比如说圆弧显示不正确。为了定位这个问题，第一步使用 Xint 参数解释运行 2D demo，发现正常，那么可以确定这个 bug 与 JIT 相关。

因为错误的现象没有涉及到本地代码，使用 GDB 也无法调试，在出现问题时，不会引起 Java 虚拟机的崩溃，也没法从报表上查找错误的原因。对于 Sun 的 HotSpot 虚拟机而言，所有方法最开始都是解释执行的，但方法执行或者循环体循环次数超过一个阀值，默认是 1500，才开始编译执行。错误的产生，肯定是由于某一个或者几个正在解释执行的方法被编译后，转为编译执行才触发的。关键是找到哪几个方法被编译执行后正好触发了错误，为了做到这点，工作中尝试修改了这个阀值，使程序运行过程中，人工可以观测到 2D demo 从正常到不正常一瞬间的变化。

利用-XX: +PrintCompilation 选项运行程序，可以得到当前被编译的方法的方法名，通过多次记录 2D demo 程序从正常到不正常变化的一瞬间正在编译的方法，可以得到一个统计，统计中经常被观测到的方法，可能就是 JIT 编译产生编译错误的地方，但由于这一瞬间编译的方法非常多，通过这种方法最后统计出的只是一个范围，不能精确到特定方法。

为了定位得更清楚，编译了X86 架构的调试版Java虚拟机，分别在X86 和龙芯上运行，并打印出中间代码，通过对比中间代码没问题，才转为察看龙芯上最后生成的本地代码。通过这样的步骤，最后发现从Java程序中调用参数为浮点类型的本地函数时，传递参数的方式不符合MIPS32 的ABI规范[10]，修正传参数的方式后，这个bug得以解决。

3.2 Java 虚拟机的性能分析和优化

这一节主要论述对 Java 虚拟机的性能分析以及优化。Java 虚拟机的性能由 Java 虚拟机内部各模块的执行性能以及动态生成的代码的执行性能体现出来，对于 Java 虚拟机的执行性能主要体现在解释器 ，JIT 编译器， 垃圾回收等，由于垃圾回收的实现与平台无关，垃圾回收性能的提高主要依赖于算法的提高，在我们的分析与优化中暂不做考虑；而解释器，JIT 编译器，运行时环境有很大一部分内容平台相关，对这一部分的优化，不但可以在性能上有比较显著的提高，而且可以在后续版本的移植中使用到现在优化的成果，所以，对这一部分的优化和分析成为工作的重点。

通过优化工作，Java 虚拟机性能提高幅度比较显著。

3.2.1 工作集和性能测试工具

SPEC JVM98 是SPEC组织发布的，用来衡量Java虚拟机性能的一个测试工具。它由由 7 个不同的测试组成，每个测试代表某一类的应用这些Java程序由SPEC组织仔细从各类应用程序中挑选出来。能够反应Java真实应用中的整数，浮点，库函数调用，I/O等性能。在Java虚拟机的性能分析中被广泛使用。JVM98 不仅可以作为基准测试程序来衡量Java虚拟机的性能，也可以作为典型的Java程序用作性能分析的工作集合，JVM98 的 7 项测试各有侧重点[11]：

- _227_mtrt 实现了一个光线跟踪算法，主要是浮点操作；
- _202_jess 是一个 Java 交互式脚本系统，利用一个数据集合和规则来解迷，主要是整数操作，没有浮点操作；
- _201_compress 使用 LZW 算法压缩，解压缩数据，主要是整数和字符操作，访存操作；
- _209_db 主要是数据库访问操作的集合，主要是的操作时排序算法，同时有大量的同步操作；
- _202_mpegaudio 主要是解码 mp3 格式文件，这个解码的过程包含大量的浮点操作；
- _202_jack 从一个规范文件产生 Java 字节码，这个过程的实现使用了很多处理异常操作；
- _213_javac 是一个 Java 编译器，主要是大量的字符串操作。

在本章，主要利用 JVM98 做为测试集合和性能分析基准程序。

为了分析 Java 虚拟机的性能瓶颈，我们采用了多种方法：

Sun HotSpot Java 虚拟机在开发中加入了一些对 Java 程序执行情况的统计功能，比如说可以统计 JIT 编译器各阶段所花费的时间，可以统计垃圾回收各阶段花费的时间等，此外，还提供了 JVM TI(JVM Tool Interface)接口，方便用户扩展监视和控制 Java 虚拟机运行的能力，并提供了一个基于此接口的一个分析工具：HPROF.。使用它可以在 Java 程序级别分析性能的瓶颈。

但用于分析Java程序执行瓶颈的工具还不足以定位出虚拟机层次的性能瓶颈，还需要使用针对本地机器代码的性能分析工具。目前，在本地机器代码级别做性能采样分析的工具很多，但绝大多数都不能支持“龙芯”平台。通过对一系列工具的调研，Gprof，Oprofile等工具可以比较方便的移植到“龙芯”平台，但对于Gprof，由于其使用Linux系统SIGPROF信号驱动的机制与Java虚拟机自定义的信号处理函数冲突，并且对多线程程序的分析存在缺陷，不适合用做分析Java虚拟机的性能。Oprofile使用CPU内部的性能计数器来统计性能，不需要重新编译程序，也不会严重降低程序的性能，它在时钟中断发生时采样PC寄存器的值，结合当前正在运行的任务的信息，转换成可执行文件加上相对于文件的偏移的形式，这样便能够定位出特定函数的执行次数。从而找出虚拟机的性能瓶颈[23]。

3.2.2 编译工具带来的优化

编译工具对程序的影响巨大，不同的编译工具编译同样的源码得到的可执行程序，在性能上可能差距很大。之前的Java虚拟机采用GCC3.4 编译，但到现在GCC发展到现在已经到了 4.1 版本。从GCC3.4 到GCC4.1，一个显著的变化是GCC4.1 采用了新的优化框架(optimization infrastructure)：Tree SSA(Static Single Assignment)[24]。这样，GCC能够在更接近输入高级语言程序的级别上做优化。

GCC 的前端在接受了输入的源程序之后，经过分析器(parser)处理得到 Parse Tree（通常是一种抽象语法数，AST, Abstract Syntax Tree），根据这个 Parse Tree 生成程序的 RTL(Register Transfer Language)表示，然后在 RTL 表示的基础上进行优化处理，然后生成相应的目标代码，如下图 3-1 所示。但是 RTL 表示是一个相当接近底层的表示，也就是说它更接近目标代码，适合进行目标相关的优化工作，比如寄存器分配等等。然而，很多的优化转换工作需要更高层的程序信息，比如数组引用、数据类型、控制流结构等等，这些很难用 RTL 表示，或者无法用 RTL 表示。

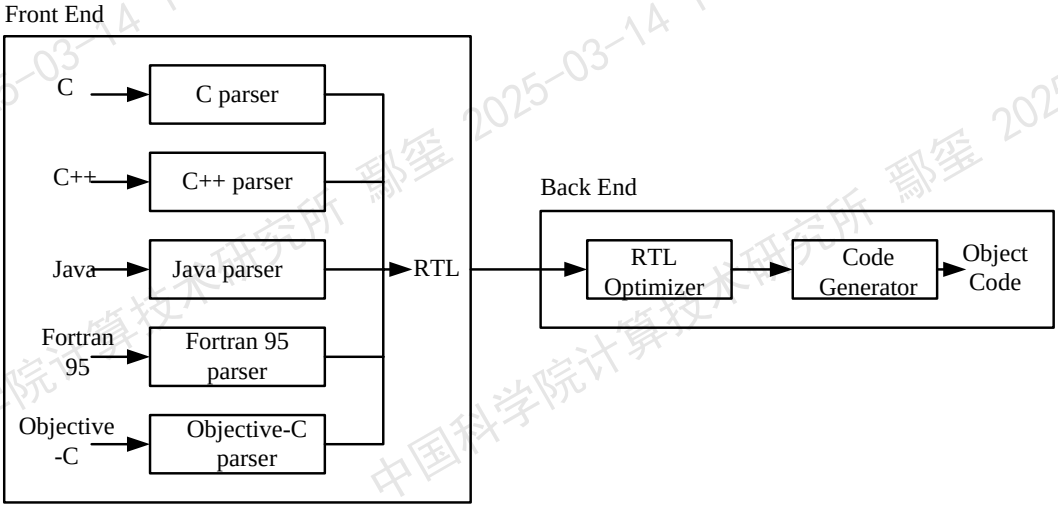


图 3-1 GCC3.4 的编译流程

GCC 4.1 编译流程比 GCC3.4 多了一级中间代码以及对中间代码的优化，如图 3-2 所示。在得到 GIMPLE Tree 之后，经过 GIMPLE optimizer 和 GIMPLE expander 的处理，得到了程序的 RTL 表示。把程序转换成 SSA 形式和在此基础上进行优化就在这个过程中完成的，在 GIMPLE 和 RTL 之间是树优化 (Tree Optimization) 的过程，主要完成这几个功能：

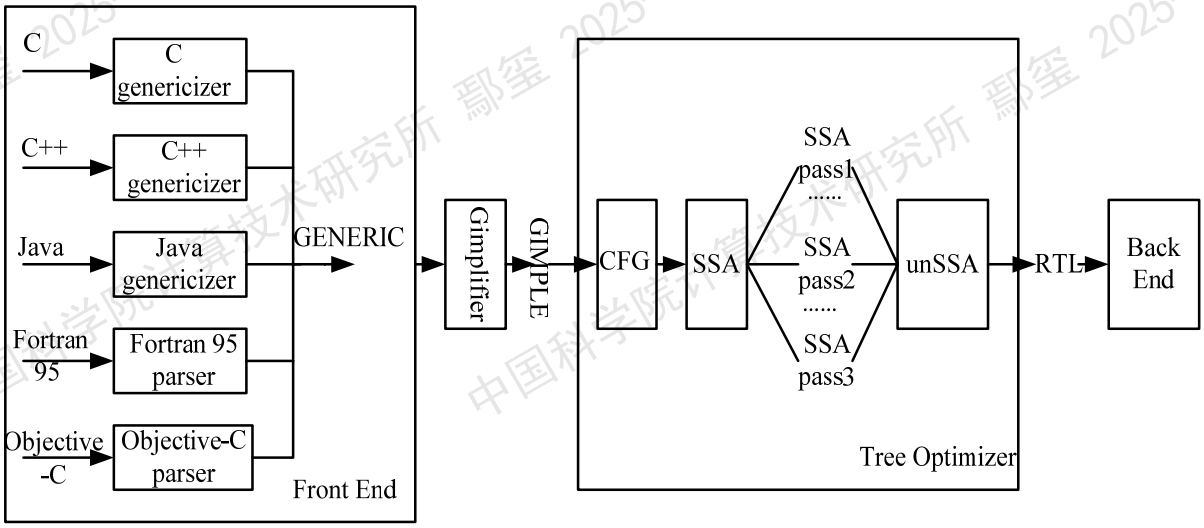


图 3-2 GCC4.1 的编译流程

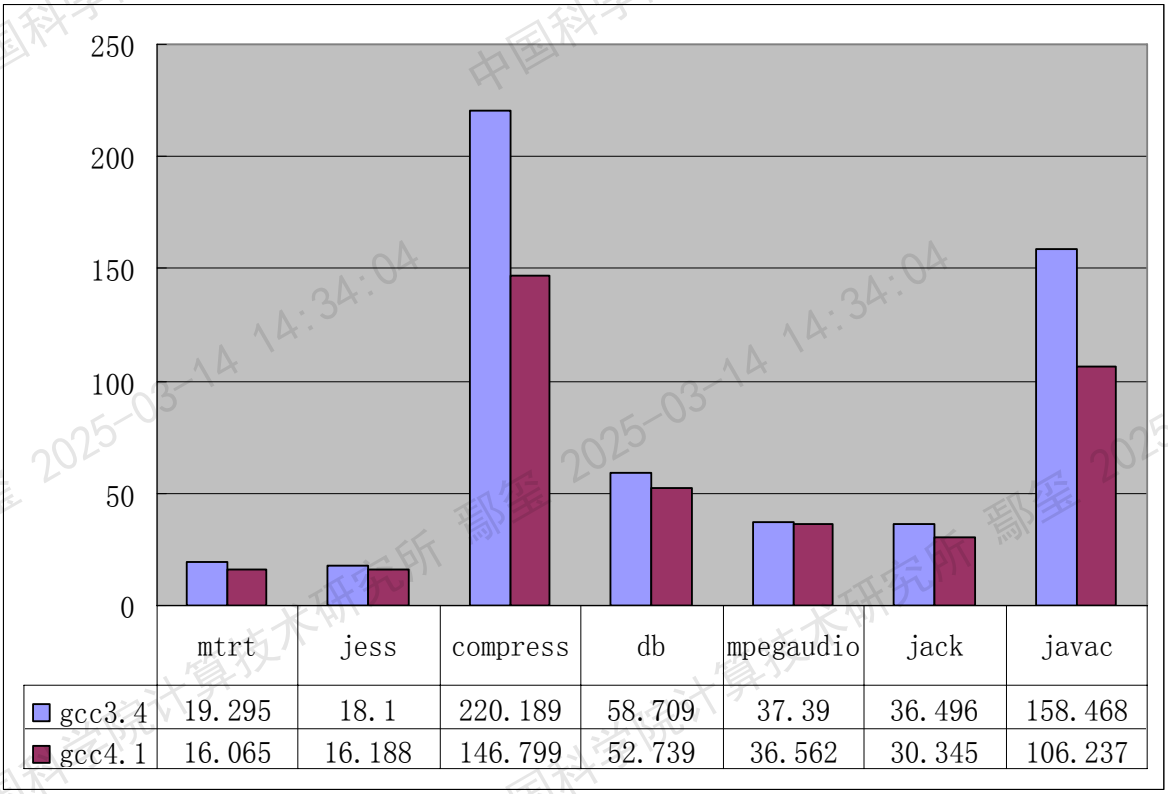
➢ 生成一个控制流转换图 CFG(Control Flow Graph)

- 将 GIMPLE Tree 转换成基于 SSA 形式的 Tree
- 进行 Tree SSA 的优化
- 将优化后的基于 SSA 形式的 Tree 转换成 RTL 接口所能识别的非 SSA Tree 的形式。

这个新的编程框架在 GCC4.0 中引进，在 GCC4.1 中发展成熟。从评测效果来看，GCC4.1 与 GCC3.4 相比，对访存类的应用程序提升比较明显，但对整数计算类的应用程序反倒有性能损失。“龙芯”属于 RISC 体系结构，其访存操作非常的多，所以，“龙芯”平台的应用软件从 GCC3.4 移植到 GCC4.1 会得到比较显著的性能提升。

Java 虚拟机从 GCC3.4 移植到 GCC4.1，利用 JVM98 测得结果如图 3-3 所示，图中纵坐标表示执行时间，单位为毫秒。

从图中可以看出来，从 GCC3.4 移植到 GCC4.1 之后，性能有了明显的提升，特别是 Compress 和 Javac 这 2 项，性能提高了大概有 49%，因为这 2 个程序都有大量的字符串操作，而字符串操作基本上通过调用 Java 虚拟机的服务函数完成，而移植到 GCC4.1 主要提升了 Java 虚拟机的执行性能。而 mpegaudio 的时间主要耗费在动态生成的代码上，Java 虚拟机执行性能虽有所提高，但动态生成代码的算法没有改变，所以提升有限。



	mtrt	jess	compress	db	mpegaudio	jack	javac
gcc3.4	19.295	18.1	220.189	58.709	37.39	36.496	158.468
gcc4.1	16.065	16.188	146.799	52.739	36.562	30.345	106.237

图 3-3　GCC3.4 和 GCC4.1 编译的 Java 虚拟机性能对比

从 GCC3.4 移植到 GCC4.1 的过程中，对源代码主要做了 4 类修改：

- friend 方法的内联，GCC4.1 中不再默认支持 friend 方法的内联，解决的办法是编译时添加一个参数：-ffriend-injection；
- 添加某些地方类，方法的前向声明：GCC4.1 在使用类和方法时，需要知道它们

的类型；

- 去掉定义类的方法时前面冗余的类名，不然 GCC4.1 的编译通不过；
- 某些地方的指针操作更加严格，不再支持指针的强制转化，有指针强制转换的地方应该改为隐式转换；

移植到 GCC4.1 主要提升的是 Java 虚拟机执行部分的性能，对于运行时动态生成代码的性能提升不起作用，如果一个 Java 应用的时间在 Java 虚拟机服务调用上耗费时间比较大，那么对这个 Java 应用的性能提高会比较显著，如果主要时间花费在动态生成的代码上，这项修改带来的效果有限。

3.2.3 优化 JIT 编译器的瓶颈

从 JVM98 的分析来看，Java 虚拟机解释执行和混合执行的性能差距在 10 倍左右，JIT 编译器对 Java 程序运行得性能影响巨大。所以，龙芯上 Java 虚拟机性能的提高，关键是要考虑提高 JIT 编译器性能。为了找出 JIT 编译器的性能瓶颈 ，用-XX+CITime 参数运行 JVM98，得到了 X86，龙芯上 JIT 编译器编译时各阶段的时间分配.对比。如图 3-4 所示。

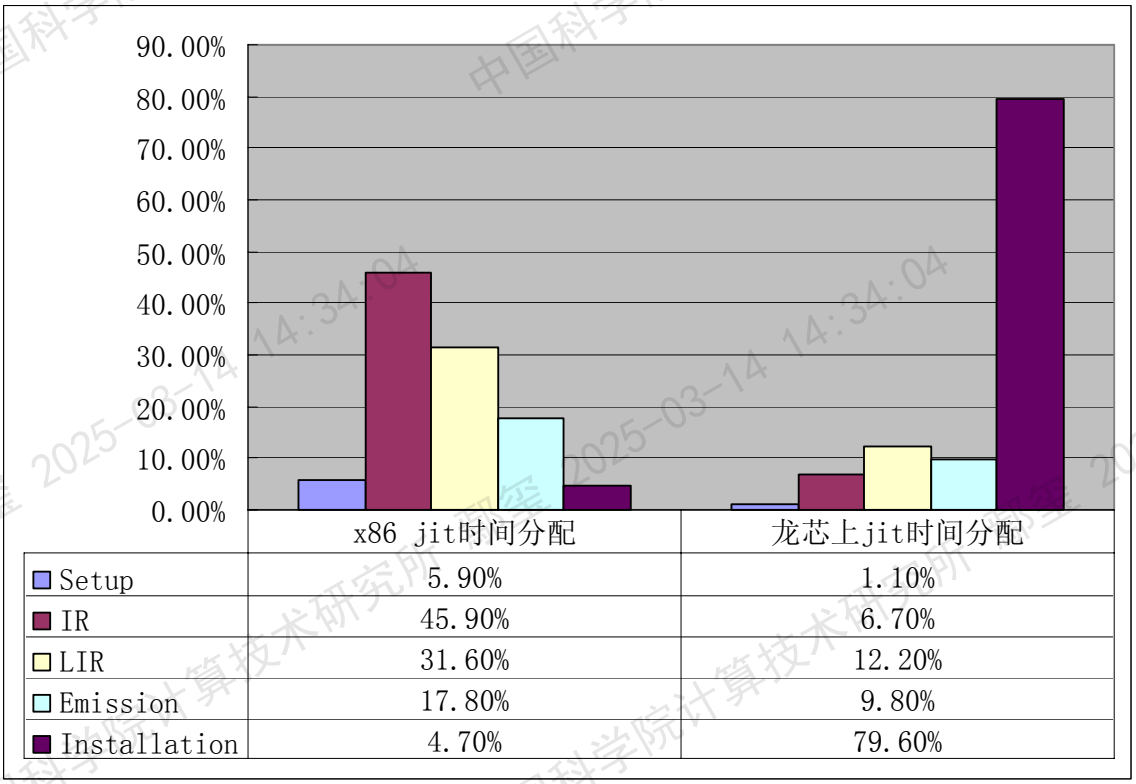


图 3-4 X86，龙芯上 JIT 编译器编译的时间分配

从图中可以看出，X86 上，JIT 编译过程的时间主要耗费在 IR 阶段，这一阶段主要完成的是扫描字节码，生成代码块，并且用中间代码填充代码块，其中包含了方法内联，公共子表达式消除，空指针检查消除等优化，这个阶段的时间占用了整个编译时间的 45%以上。其次是 LIR 阶段，这个阶段完成的任务是根据 IR 中间代码生成接近低层中间代

码，包含了寄存器分配，并做了简单的窥孔优化，这个阶段占用了整个编译时间的 30%以上。相比之下，“龙芯”平台的 JIT 编译器在 IR 和 LIR 阶段总共才占用了不到 19%的时间，而代码的安装(Install)阶段占用了几乎 80%，这个阶段完成的主要功能是代码拷贝，重定位，对比之下，这个阶段在 X86 平台之占用了不到 5%所以 JIT 编译器性能的瓶颈主要在代码安装阶段。代码安装阶段 Java 虚拟机主要的操作是访存操作，从硬件上说，“龙芯”平台的访存速度的确比 X86 慢，但差距并没有几倍。这说明：在这个阶段，软件上有对“龙芯”平台不合适的地方，即这个地方有比较大的优化空间。

通过分析 JIT 编译时代码安装的过程，得知：在安装代码时，假如 CPU 需要运行刚才生成的代码，如果指令 cache 中正好有该地址的旧的数据，CPU 会从指令 cache 中得到错误的数据，这样会引发非法指令异常，导致 Java 虚拟机崩溃，所以在 Java 虚拟机执行过程中需要保持指令 cache 和内存对应位置数据的一致性。X86 结构的处理器从硬件上实现了这个功能。但对于“龙芯”处理器，这个一致性需要靠软件来实现，这样，在生成代码阶段，“龙芯”平台的 Java 虚拟机比 X86 平台的版本多了一个刷新 cache 的操作。

Java 应用程序在执行过程中，如果编译操作频繁，刷新 cache 的操作的执行将非常多。因为刷新 cache 操作是一个特权指令，则这样的操作必然反映到某条系统调用上，通过研究，得知，最终直接刷新 cache 的系统调用为：blast_scache32。为了查明这个系统调用的执行在系统中占用的时间比，我们采用了 Oprofile 作为分析工具，用 JVM98 作为工作集，对 Java 的执行，分析的结果如下表所示。Oprofile 以 CPU_CLK_UNHALTED 为采样事件，每 30000 个时钟周期采样一次。表 3-1 所示是 Java 运行时，调用的函数的时间分配。samples 表示采样到的次数，%表示在执行中所占的百分比，image name 表示函数所在的段，symbol name 表示执行函数名。其中 image name 中出现 anon 是因为 Java 执行编译出来的本地代码没有符号信息。

表 3-1 Java 进程中 CPU 时间分配表（JIT 优化前）

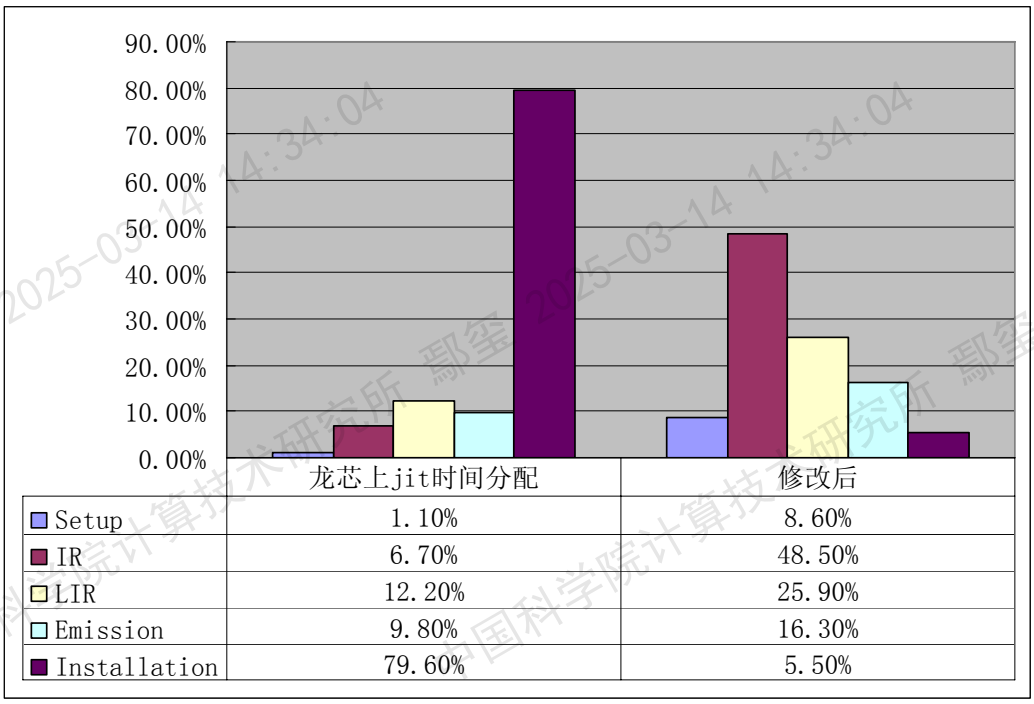
samples	%	image name	symbol name
4424293	52.4822	vmlinux	blast_scache32
749341	8.8889	anon(tgid:24603 range:0x2e438000-0x30258000)	no symbols
700778	8.3128	anon(tgid:24658 range:0x2e258000-0x2e448000)	no symbols
346393	4.1090	anon(tgid:24682 range:0x2e258000-0x2e450000)	no symbols
203487	2.4138	anon(tgid:24713 range:0x2e258000-0x2e500000)	no symbols
143069	1.6971	anon(tgid:24593 range:0x2e3f8000-0x30258000)	no symbols
125455	1.4882	anon(tgid:24700 range:0x2e258000-0x2e488000)	no symbols
101296	1.2016	anon(tgid:24682 range:0x2e450000-0x30258000)	no symbols
80575	0.9558	anon(tgid:24713 range:0x2e500000-0x30258000)	no symbols
77227	0.9161	vmlinux	__do_softirq
……	……	……	……

从表中可以看出，blast_scache32 的执行，占用了整个 Java 进程运行时间的 50%以上，如果能够有效地减少这个操作的时间，将显著的提高 Java 虚拟机的性能。

目前，刷新 cache 的操作是每次刷新全部的 cache，通过对 Java 虚拟机工作原理的分析，得知，代码安装时刷新 cache，并不需要刷新全部的 cache，只需要刷新刚才更新的代码，所在的地址范围，在 cache 中的项即可，而这个范围，在大多数时候只有 2~3 个字。原来刷新全部 cache 的方案浪费了大量的 CPU 时间。

为了减少 cache 操作的时间，修改 cache 操作方案，每次只刷新一个小的地址范围。但这样的修改带来一个难点：如何去精确的确认这个地址范围。这样就需要仔细的记录代码安装时生成的代码的地址范围，通过在 LIR 中间代码转换到本地代码过程中记录该转换生成的代码的地址范围，记录重定位后新的地址范围，更新跳转语句后，跳转语句的地址范围等，最后得到一个应该刷新的地址范围，修改 cache 操作，把安装代码时刷新整个 cache 的操作改成了仅仅只刷新一个地址范围。

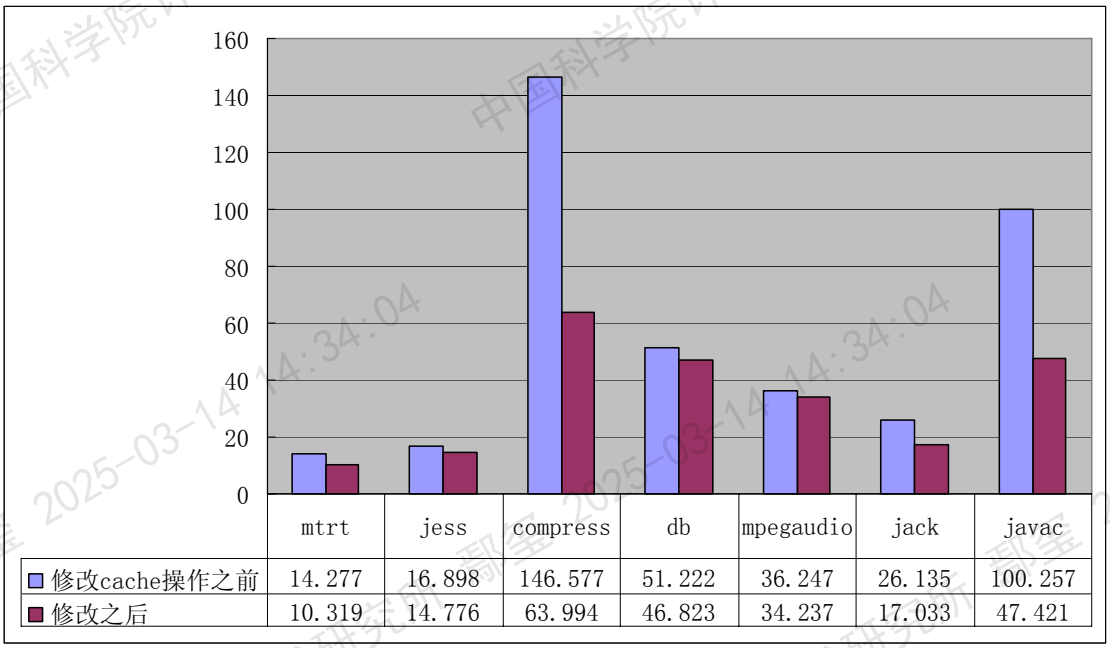
经过若干调试，确认修改达到稳定的效果后，再次利用 Oprofile 分析，得知 blast_scache32 系统调用所占用的时间从 52.48%减少到了 0.02%，这说明这个修改减少了绝大多数无效的 cache 操作。此时 JIT 各阶段所占用的时间比如图 3-5 所示。与修改之前比较，修改后代码安装阶段的时间明显减少。所占用的时间从 79.6%下降到了 5.5%，这说明代码安装阶段的瓶颈基本上得以消除。同时，LIR 阶段和本地代码生成阶段(Emission)的时间也显著减少，因为这 2 个阶段也包含了一些对 cache 的操作。



	龙芯上jit时间分配	修改后
Setup	1.10%	8.60%
IR	6.70%	48.50%
LIR	12.20%	25.90%
Emission	9.80%	16.30%
Installation	79.60%	5.50%

图 3-5 修改 cache 操作后 JIT 编译各阶段时间分配

再次利用 JVM98 分析这个修改所带来的影响如图 3-6 所示，图中纵坐标表示执行时间，单位为毫秒。



	mtrt	jess	compress	db	mpegaudio	jack	javac
修改cache操作之前	14.277	16.898	146.577	51.222	36.247	26.135	100.257
修改之后	10.319	14.776	63.994	46.823	34.237	17.033	47.421

图 3-6 修改 cache 操作前后 JVM98 的分值变化

从图中可以看出，刷新 cache 操作的修改对性能提高明显。Compress 和 Javac 这两项的性能提高了 110%以上。因为这 2 项测试有大量的对代码块的操作，这些操作需要大量的 cache 操作，而 mpegaudio 这项测试提高有限，因为这项测试主要是解码测试，Java 虚拟机运行的指令局部性很强，对代码块的操作不多也不多，所以改变 cache 操作带来的性能提升不明显。

由于操作 cache 在系统空间进行，频繁的操作势必会增加系统调用的开销，为了研究这个开销给 Java 虚拟机带来多大的性能影响，我们修改了内核，使得在用户空间也可以直接调用 cache 操作指令，然后修改 Java 虚拟机，去掉系统调用，直接嵌入汇编调用 cache 指令。然后用 JVM98 测得性能变化如图 3-7 所示，图中纵坐标表示执行时间，单位为毫秒。

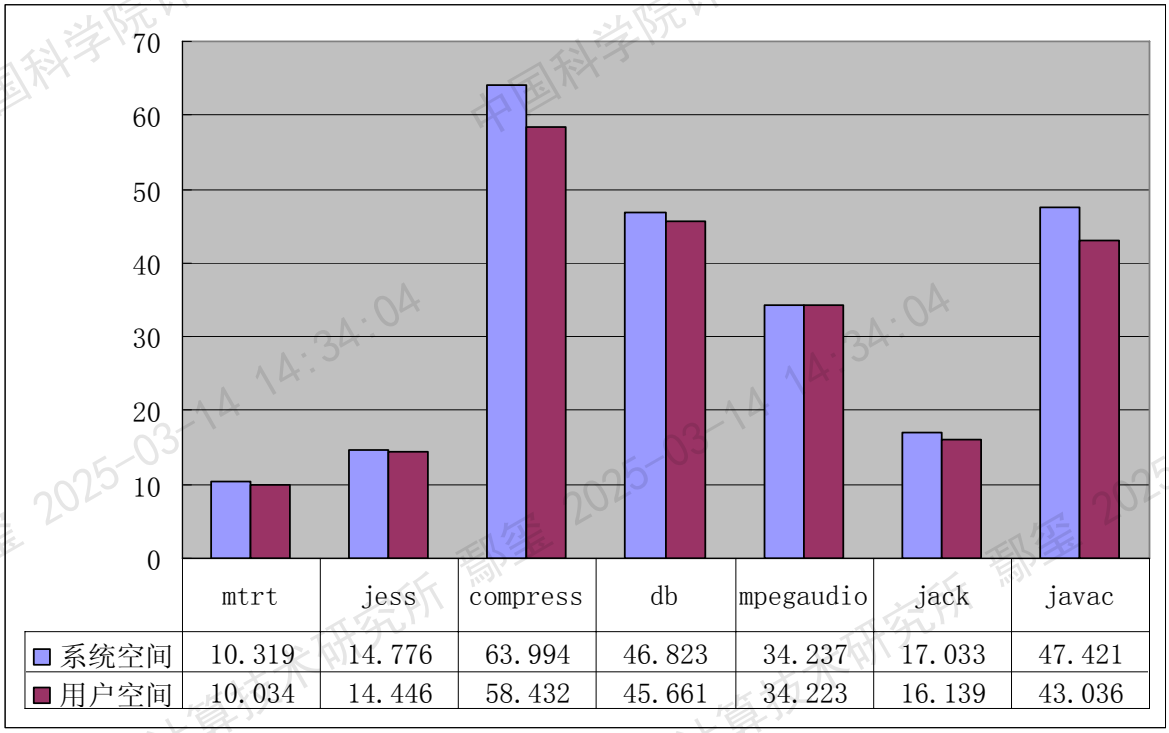


图 3-7 cache 操作放在用户空间与系统空间 JVM98 的分值变化

从图中可以看出，修改内核把 cache 操作从系统空间提升到用户空间能在性能上带来 5%左右的提高，这个效果并不明显，反倒把对 cache 操作直接暴露给用户，带来了安全隐患，所以，最后，我们没有采用这种优化方案。

总的来说，cache 操作的改进，对 JIT 编译器的性能提高比较显著。这个修改主要有利于编译较多的场合，对于编译较小的场合，比如说很简单的 applet，这个改进带来的性能提升有限。

3.2.4 解释器的优化

解释器的性能在一定程度上也能反映整个 Java 虚拟机的性能，特别是在某些小的应用程序启动速度上。在 X86 架构上，对 JVM98 的解释执行和混合执行所需时间如图所示。

在 X86 上对 JVM98 的解释执行和混合执行的对比如图 3-8 所示，不同的测试，执行时间比差别很大，mepegaudio 相差达 21 倍，因为这个程序是局部性比较强，局部执行的部分在混合模式下是编译执行，“编译执行”占了不小的比例。相差最小的是 Jack，只有 4 倍的差距，因为这项测试主要是异常处理，这一部分是解释执行和编译执行共享的。从 JVM98 的结果得出，平均起来，解释执行所需要的时间是混合执行的 10~11 倍。

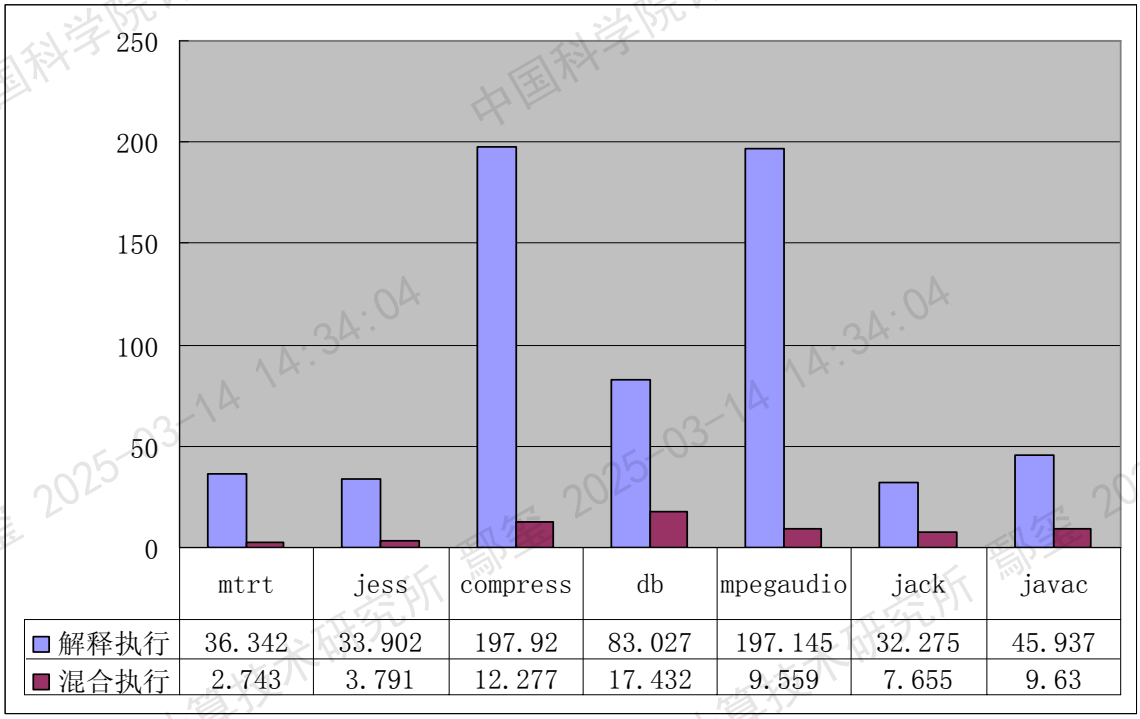


图 3-8 X86 上解释执行和混合执行 JVM98 分值变化

在“龙芯”平台上解释执行与混合执行的对比如图 3-9 所示。

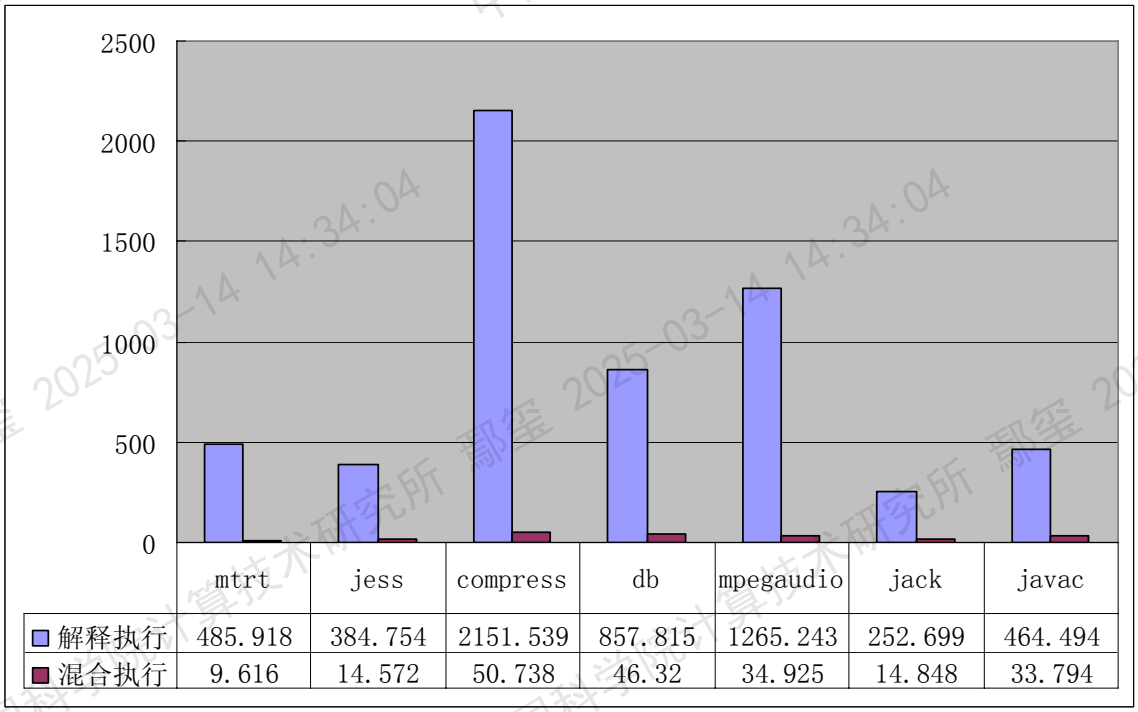


图 3-9 龙芯上解释执行和混合执行 JVM98 分值变化

解释执行和混合执行性能差别比 X86 平台要大很多。最大的 Mtrt 测试高达 51 倍，差别最小的 Javac 也差别 14 倍，平均起来高达 27~28 倍，对比 X86 版本，龙芯上解释执行的性能相对混合执行过差。

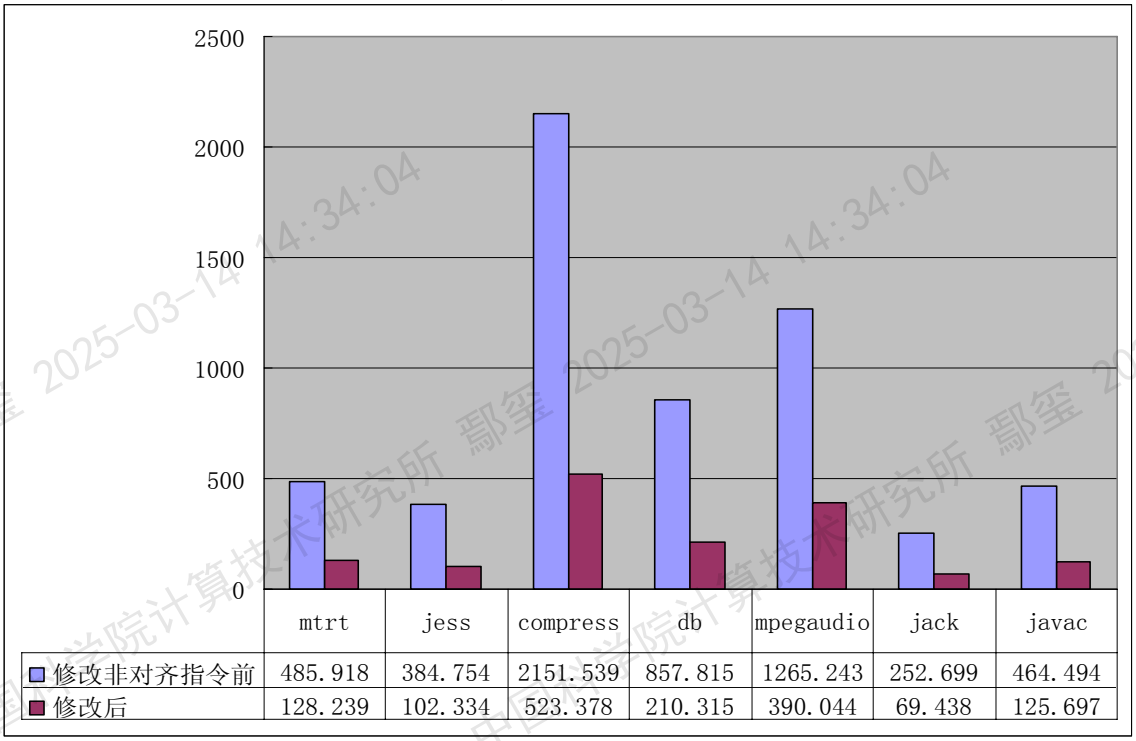
为了分析解释器的性能瓶颈，我们采用了多种办法。通过分析解释器的工作过程，

推测执行时间主要花费在执行翻译出来的本地代码上。但在我们移植过程对解释器的结构没有做大的修改，对每一个条字节码的翻译，生成的本地代码也尽量精简。虽然解释执行基本上是堆栈操作，推测原因是访存操作比较多，不利于“龙芯”等 RISC 结构 CPU 的性能发挥。

通过反复用 Oprofile 分析，发现 Java 进程中 do_ade 这个函数调用执行次数比较多，时间占用到整个进程执行时间的 2%左右。do_ade 主要完成处理非对齐地址访问的异常。这说明解释器对“字节码”解释的结果中，存在对非对齐地址的访问。而对非对齐地址的访问会触发操作系统的异常，从而进入相应的异常处理函数，这个异常处理的过程，消耗的时间一般是几千条指令，如果减少对非对齐地址的访问，解释器性能的提高应该非常显著。

通过反复的调试，验证，修正了这个问题：如果发现“半字访问”的地址不对齐，则把“半字访问”改成对 2 个字节的访问，如果发现对字的访问不对齐，使用 2 条“不对齐字访问指令”组合代替简单的字访问。

不对齐地址问题出现的根源在于 Java 虚拟机协议对常数区并没有规定需要按照地址对齐，这样规定也是出于节约空间考虑。对非对齐地址的访问对 X86 结构而言，是没有影响的，因为 X86 结构会从硬件上处理这个问题，但对于“龙芯”等 RISC 结构的 CPU 而言，这样的操作通常会导致异常，需要耗费大量的 CPU 时间做异常处理。



	mtrt	jess	compress	db	mpegaudio	jack	javac
修改非对齐指令前	485.918	384.754	2151.539	857.815	1265.243	252.699	464.494
修改后	128.239	102.334	523.378	210.315	390.044	69.438	125.697

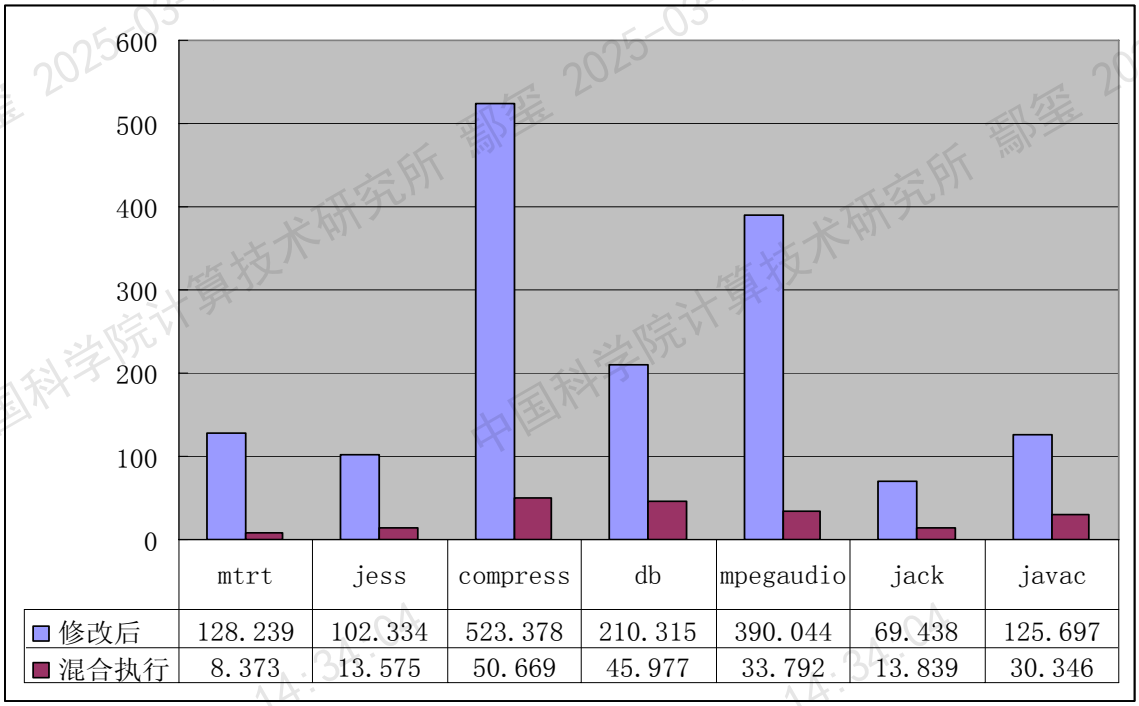
图 3-10 处理对“非对齐地址”访问前后 Java 解释器性能对比

值得注意的是对不对齐地址的访问，只有在解释执行时才出现。因为在编译执行过程中，Java 中的变量，参数等要么放到以字对齐的堆栈中，要么放到了寄存器中，这样由 Java 程序产生的代码就不存在对不对齐地址访问的问题，而将数据放到堆栈，寄存器

的过程由 C++高级语言完成，GCC 编译器在编译时作了处理，不会生成访问“非对齐地址”的指令。

修改不对齐指令之后，解释器的性能有了大幅的提升，使用 JVM98 解释执行，得到的结果，如图 3-10 所示。平均起来执行时间有将近 3 倍的差距。

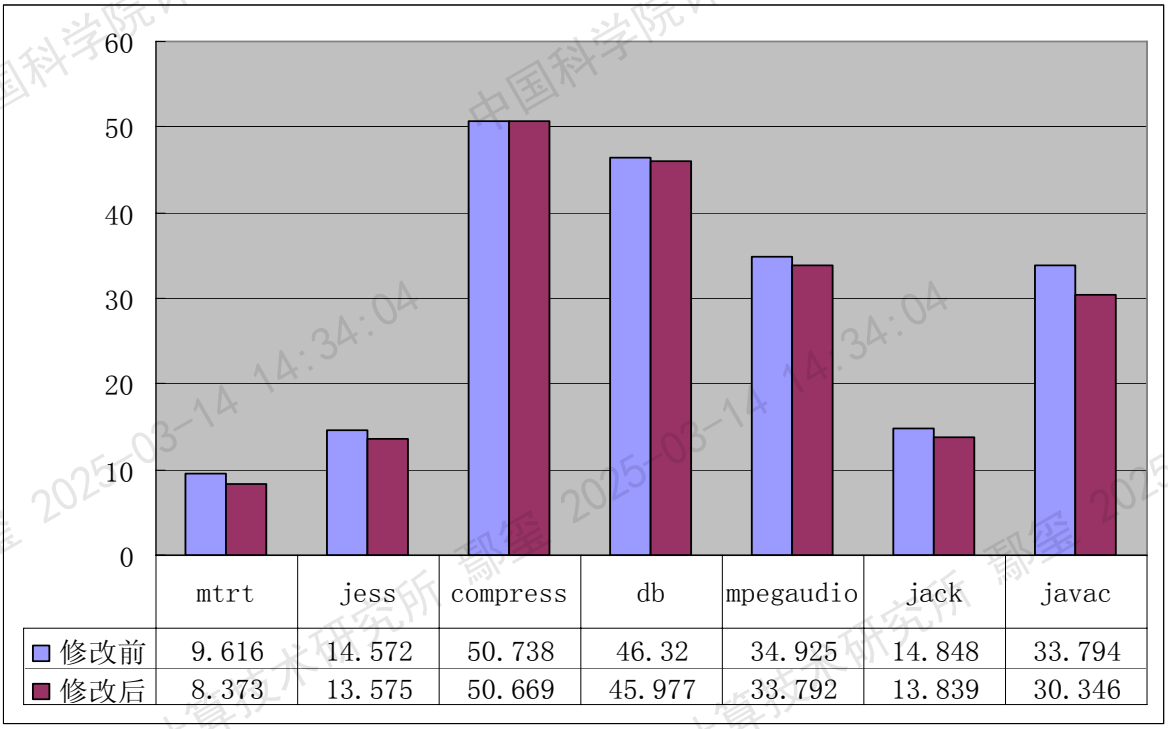
此时，龙芯上解释执行与混合执 JVM98 的对比如图 3-11 所示。性能差距最大的 Mtrt 也只有 15 倍的差距。最小的 Javac 只相差 4 倍，平均起来差距 8~9 倍，这个比例现在略小于 X86 版本了，运行时间的比例基本上合理了。这说明解释器的性能瓶颈基本上消除了。



	mtrt	jess	compress	db	mpegaudio	jack	javac
修改后	128.239	102.334	523.378	210.315	390.044	69.438	125.697
混合执行	8.373	13.575	50.669	45.977	33.792	13.839	30.346

图 3-11 处理对非对齐地址的访问后解释执行，混合执行性能对比

消除解释器的性能瓶颈后，JVM98 混合执行性能也有所变化，如图 3-12 所示。



	mtrt	jess	compress	db	mpegaudio	jack	javac
修改前	9.616	14.572	50.738	46.32	34.925	14.848	33.794
修改后	8.373	13.575	50.669	45.977	33.792	13.839	30.346

图 3-12 处理非对齐指令前后 JVM98 执行性能对比

JVM98 测试中 Mtrt，Javac，Jess，Jack 这 4 项测试提高比较多，在执行时间上分别减少了 13%，10%，10%，7%平均起来，因为这 4 项相对而言，解释执行的部分较多，mpegaudio 的性能提高有限，因为这个测试主要是编译执行，解释器性能的改进带来的影响不大。JVM98 测试结果表明，消除解释器对非对齐地址的访问后，JVM98 执行时间平均减少了 6.3%，综合性能平均提高了 10%左右。在实际应用中，解释器性能提高带来的影响，主要表现在 Java 程序的启动速度上，特别对于一些小的 Java 程序，applet 等，启动速度提高明显，即使对永中 office 这样的大规模的 Java 程序，启动速度也能有一定的提高，用秒表测量了修改前后永中 office 第一次启动，第一次打开 doc 文档的时间。测量得知，永中 office 第一次的启动速度，平均从原来的 13.33 秒减少到现在的 10.70 秒，减少幅度为 19.7%，第一次打开一个同样大小的 doc 文档的时间从原来的 11.64 减少到现在的 9.74 秒，减少幅度为 16% 。

总的来说，消除对非对齐地址的访问对解释器性能的提高有显著的影响，这个优化带来的好处，主要表现在 Java 程序的启动速度和小 Java 程序的运行上。对于比较大的程序，由于其“编译执行”占了不少的比例，带来的影响有限。

3.2.5 其它优化措施

HotSpot Java 虚拟机采用了的是一种比较简单的自适应编译执行方式。虚拟机在执行 Java 程序时，对每个方法都是解释执行，但是在执行过程中对每个 Java 方法的执行次数或者方法内循环体循环次数进行统计，如果某方法的执行次数或循环体循环次数超过了一个阀值，就引发了对这个 Java 方法的编译，编译完成后，执行编译后生成的代码，

代替之前的解释执行方式。这个参数与解释器性能，JIT 编译器性能等相关。

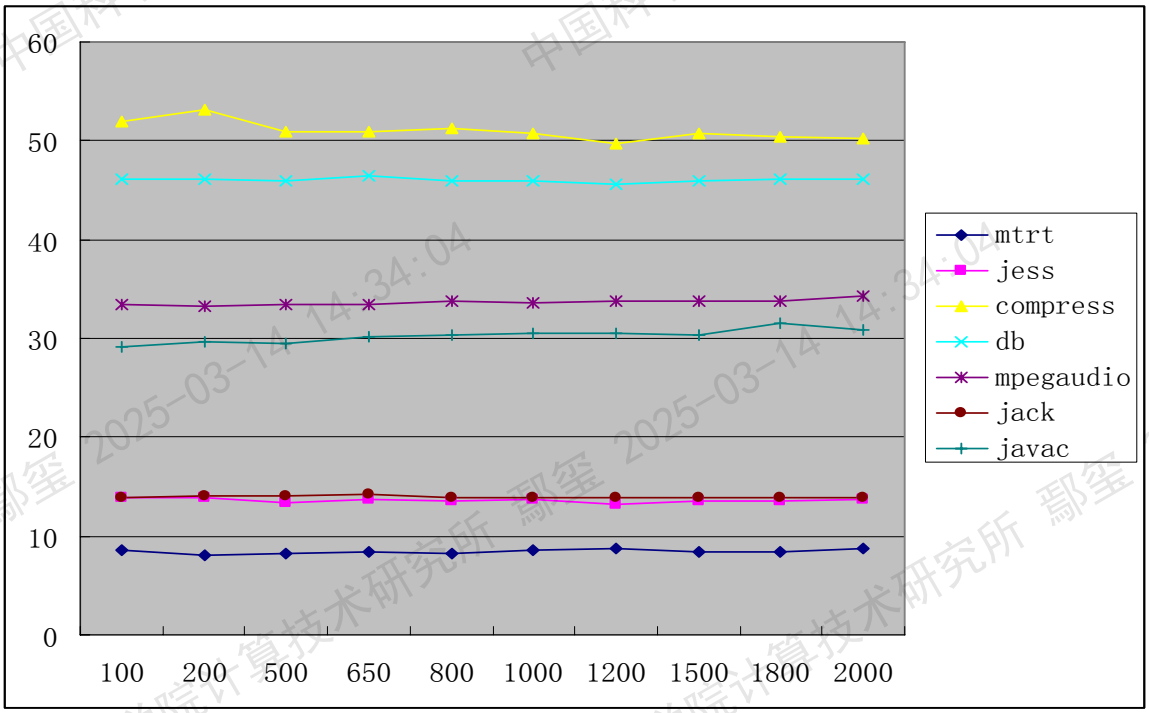


图 3-13 选择不同的阀值执行 JVM98 的结果

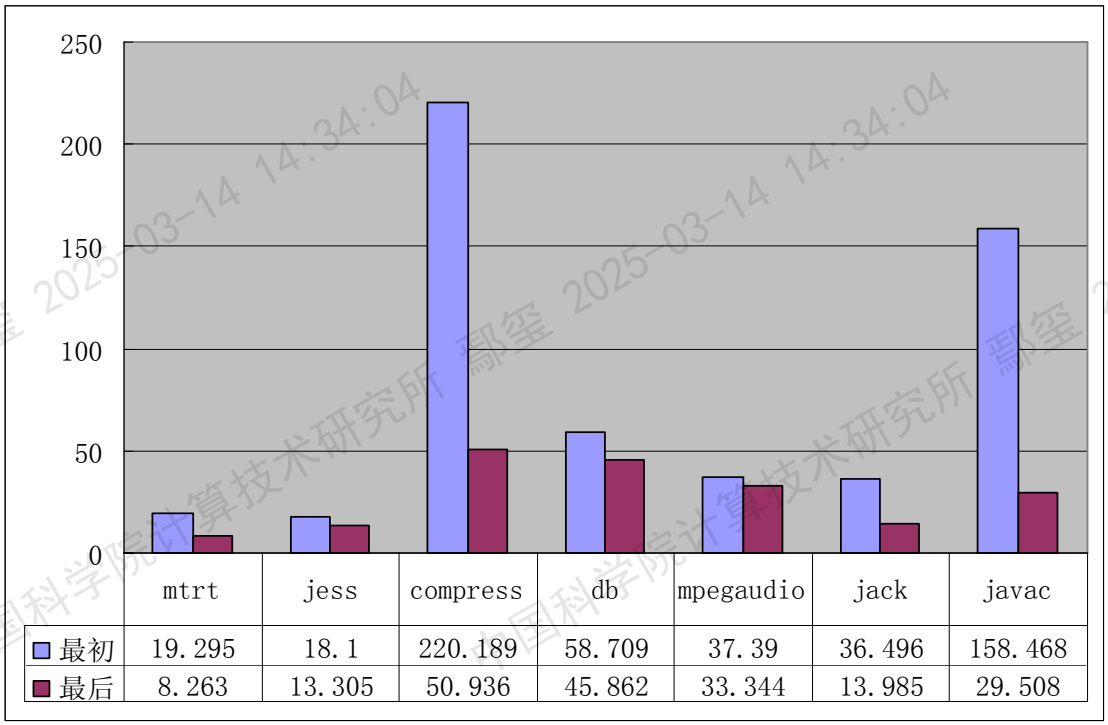
X86 版本客户版 Java 虚拟机默认的阀值是 1500，即一个方法解释执行或者循环体循环 1500 次后，就会引发对方法的编译，1500 这个阀值是 Sun 公司对 X86 平台上 Java 执行情况得出来的较优的参数，但是对于“龙芯”平台，最优性能的阀值可能和 X86 平台有所不同，为了找出最优的阀值，我们在 Java 虚拟机源码中修改参数，设置不同的阀值编译，然后用 JVM98 测试性能，对于不同阀值下，JVM98 测试结果的波动如图 3-13 所示。纵坐标表示需要的时间，横坐标表示选取的编译阀值，在图中，我们尽量选择这样一个点：这个点对于 7 条曲线而言，都能达到一个比较好的值， 综合考虑，选择了 500 做为阀值，采用这个值，比 1500 的值，性能上平均能够提高 2%左右。

“龙芯” 对比X86 结构，一个明显的不同是通用寄存器数量多很多。但 Sun的 HotSpot Java虚拟机客户端版本主要针对X86 结构设计，移植参照X86 版本完成，导致了“龙芯”的通用寄存器利用不充分。打印出动态生成的代码统计后可以发现，每个功能块内对通用寄存器的使用一般不超过 5 个，为了充分利用多出的寄存器，在优化运行时环境时，使用了一些寄存器来缓冲一些常用的变量，避免了一些堆栈操作[2,3,5,16]，比如说使用了一个寄存器来存储Java类的局部变量区的首址，使用了一个寄存器来存储当前线程指针等，这样给Java虚拟机性能带来了大约 1%左右的提高。

3.2.6 本节总结

经过我们的优化，Java 虚拟机的性能和最初相比的提高情况如图 3-14 所示。纵坐标

表示执行时间，单位为毫秒，经过本章所述的优化工作，Java 虚拟机的性能提高了很多：Compress 和 Javac 的测试所用的时间分别减少了 79%，81%，即性能提高了 3 倍多，4 倍多，最小的 mepegadio 测试，性能也提高了 12%，平均起来，JVM98 性能提高了 168% 左右。



	mtrt	jess	compress	db	mpegaudio	jack	javac
最初	19.295	18.1	220.189	58.709	37.39	36.496	158.468
最后	8.263	13.305	50.936	45.862	33.344	13.985	29.508

图 3-14 优化前和最终优化后 Java 虚拟机性能对比

虽然目前龙芯上 Java 虚拟机的性能有了长足的发展，和 X86 平台的 Java 虚拟机，性能还存在差距。如图 3-15 所示。665m 频率的龙芯，256m 内存的系统上 JVM98 的执行时间平均起来比 2.66g 的奔四，512m 内存的系统多 2～3 倍。虽然综合硬件方面性能差距看来，JVM98 测试成绩的差距并不算大，但对于 Java 应用程序而言，与目前主流系统存在这么大的差距，对用户而言是无法容忍的。龙芯上 Java 应用性能的提高，一方面需要依赖硬件方面的提高，二方面还需要继续发掘自身的性能瓶颈，争取在不影响稳定性，兼容性的情况下，对软件的性能进行提高。

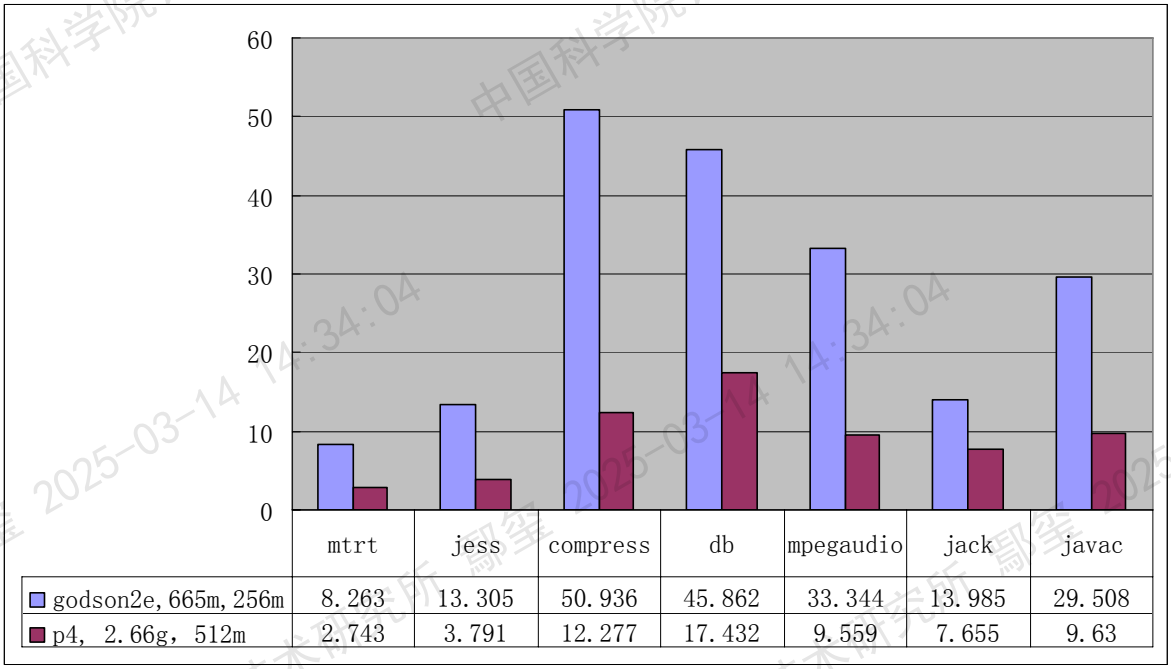


图 3-15 “龙芯”平台和目前典型 X86 平台的性能对比

表 3-2 优化后 Java 进程执行的时间分配

samples	%	image name	symbol name
850139	20.8057	anon (tgid:25785 range:0x2e4a0000-0x30258000)	no symbols
704898	17.2512	anon (tgid:25831 range:0x2e258000-0x2e4d0000)	no symbols
695861	17.0300	anon (tgid:25809 range:0x2e258000-0x2e4b0000)	no symbols
227953	5.5788	anon (tgid:25774 range:0x2e258000-0x2e498000)	no symbols
166161	4.0665	anon (tgid:25850 range:0x2e258000-0x2e520000)	no symbols
159424	3.9016	anon (tgid:25858 range:0x2e258000-0x2e578000)	no symbols
104969	2.5689	anon (tgid:25858 range:0x2e578000-0x30258000)	no symbols
61572	1.5069	libc-2.3.6.so	no symbols
39193	0.9592	anon (tgid:25768 range:0x2e430000-0x30258000)	no symbols
36972	0.9161	libjvm.so	RelocIterator::set_limits(unsigned char*, unsigned char*)
……	……	……	……

通过分析，此时 Sun 1.5 版的 Java 虚拟机的性能瓶颈主要存在于 Java 虚拟机动态生成并执行的代码上。从 Oprofile 工具分析的结果来看，动态生成代码的执行时间，占了整个 Java 进程执行时间的 72% 以上。如表所示。对动态生成的代码质量的的提高，是再次提高 Java 虚拟机性能的关键。由于 Sun 的 HotSpot 客户端版虚拟机主要面向的是交互性的小应用，强调的是程序的反应时间，编译代码时主要考虑编译的速度，对生成的代码所做的优化不多。从而导致了生成代码的低效。而服务器版的 Java 虚拟机强调运行程序峰值的性能，对生成的代码作了很多的优化，代码质量较高，但编译过程非常缓慢，

导致程序响应速度不够，不适合桌面的应用。所以，JIT 编译器还需要从结构上优化，才足以比较显著的再次提高 Java 虚拟机的性能，在下一章中将论述这个问题。

除了运行时生成的代码，Java 进程的时间在重定位上面花费不小，如表 3-2 所示，紧接着运行时代码分配时间最多的便是重定位（RelocIterator）相关的操作了，造成重定位操作如此频繁的原因主要在移植中没有采用 X86 类似的局部函数调用和返回（使用相对地址偏移），方法调用，返回均使用绝对地址，在调用运行时代码时发生重定位，方法返回时，必须改写返回地址，这个需要通过重定位来完成，如果能找到好的办法减少重定位操作将有助于提高 Java 虚拟机的性能，这是将来的一个优化点。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

第四章 HotSpot Java 虚拟机 1.6 版分析与移植

Sun 公司在 2006 年底正式发布了 JDK1.6 版本，JDK1.6 版本的开发历时 2 年，相比 JDK1.5 版本，增添了一些功能，同时由于其客户端几种关键技术的改进，在性能上提高比较大。本章主要论述新的 1.6 版 Java 虚拟机在龙芯上的分析与移植。

4.1 移植到新版本 Java 虚拟机的必要性

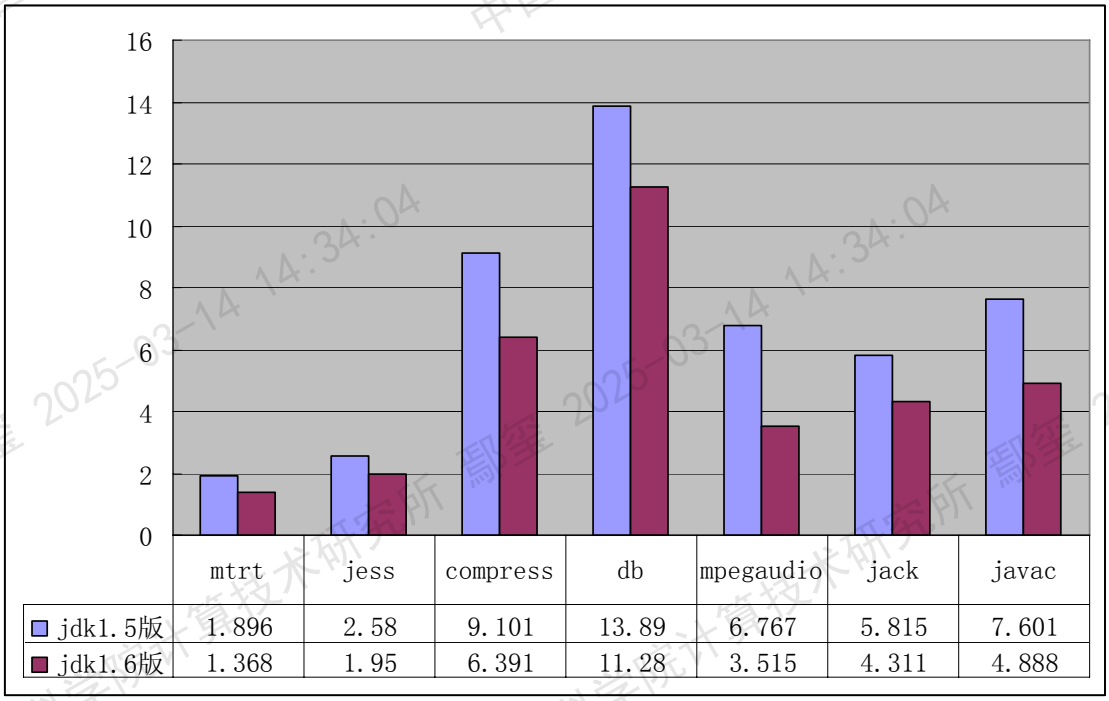
Sun在 12 月份发布了JDK1.6，代号Mustang，它增加了一些新的语言特征[25]：

- Web Services：包含了一个 Web Services 客户端实现 JAX-WS，同时如加入了 JAXB 实现和 Stax 实现，使开发 SOA 程序更加容易；
- 脚本：支持在 Java 代码中混入 Javascript 脚本，Java 6 提供了一个标准接口，理论上支持所有脚本语言。在 Java 中使用脚本语言的一个好处是更容易让用户来扩展和定制程序，尤其是一些需要经常更改业务逻辑 的程序，如终端用户可以通过在表单中输入新的业务规则脚本来改变程序的逻辑；
- 数据库：内置了 Apache Derby 数据库，这对于开发桌面应用程序来说是个不错的功能，不用用户再额外安装其它外置数据库了，同时加入了 JDBC4.0 的一些新功能。
- 新的桌面 APIs：加入了一些新的 APIs，如 Swingworker、GroupLayout 。
- 监控和管理：添加更多的诊断信息，绑定了不是很知名的 memory-heap 分析工具 Jhat 来查看内核导出；
- 编译器访问： compiler API 提供编程访问 Javac，可以实现进程内编译，动态产生 Java 代码；
- Pluggable Annotations：Java tool 和 framework 提供商可以定义自己的 annotations ，并且内核支持自定义 annotation 的插件和执行处理器；
- 桌面发布：Swing 拥有更好的 look-and-feel , LCD 文本呈现, 整体 GUI 性能的提升。Java 应用程序可以和本地平台更好的集成，Mustang 将 Java 插件技术和 Java Web Start 引擎统一了起来；
- 安全性：支持 XML 数字签名，整合了 PKI、Java GSS、Kerberos、和 LDAP。

除了增加新的特性外，新版本的 Java 虚拟机在性能方面得到了比较明显的提升。使用 JVM98 分别对 Java 虚拟机的 1.5 版，1.6 版在 X86 上的性能做了测试，测试使用同一台机器，同样的环境，执行 JVM98 的时间如图所示。从图 4-1 中可以看出，Java 虚拟机的新版本与上一个版本相比，性能提高很快，平均性能提高了 45%左右。

1.6 版本 Java 虚拟机性能的提升归功于 JIT 编译器改进，垃圾回收机制改进，同步机制改进等多方面，从移植的角度考虑，最关心的是平台相关部分的改进。在新的 Java

虚拟机中，平台相关部分最显著的变化是 JIT 即时编译器的改进，JIT 编译器最大的改进在于引进了一种新的寄存器分配算法：线形扫描寄存器分配算法。



	mtrt	jess	compress	db	mpegaudio	jack	javac
jdk1.5版	1.896	2.58	9.101	13.89	6.767	5.815	7.601
jdk1.6版	1.368	1.95	6.391	11.28	3.515	4.311	4.888

图 4-1 1.5 版 Java 虚拟机和 1.6 版 Java 虚拟机性能对比

从上一章的论述中得知，龙芯上 1.5 版 Java 虚拟机的性能瓶颈主要在于动态生成的代码的质量比较差，而代码质量的改进主要取决于 JIT 编译器的改进，在新的 Java 虚拟机中，主要改进了寄存器分配的问题，对于 RISC 结构的 CPU 而言，寄存器分配算法的对代码质量的影响巨大，新版本的 Java 虚拟机在“龙芯”移植好后，必将能够显著的提高龙芯上 Java 执行环境的性能。

4.2 JDK1.6 在龙芯上的移植分析

JDK1.6 的源代码中，hotpot 目录里的源文件编译生成 libjvm.so(product 版本)，这个动态库是 Java 虚拟机的实现，也是 JDK1.6 的核心。除了 HotSpot 目录外，其它目录的源文件都是使用 Java 或者 C++等高级语言编写，可以直接编译，对于某些目录生成的*.jar 文件，因为是 Java 的 2 进制文件，不需要编译可以直接使用。所以，对于移植而言，需要关注的只有 hotpot 目录。

HotSpot 目录下，有 4 个目录：share 目录是整个 Java 虚拟机设计的核心部分，这一部分被 Sun 公司设计为平台无关的，这个目录中定义的类的平台相关部分被剥离，分别放到了 cpu，os_cpu，os 3 个目录中：os 目录是操作系统接口部分，CPU 目录是体系结构部分代码，os_cpu 是体系结构和操作系统的接口，这 4 个目录分别代表了 Java 虚拟机的 4 个层次，从最低层的 CPU，然后通过 CPU 和操作系统的接口层到达操作系统，再通过操作系统和应用程序的接口到达应用程序，层次感非常清晰。

share 目录主要由下列子目录构成：

- Adlc：根据平台描述文件生成平台相关的*.hpp，*.cpp 文件，这个目录主要用来支持服务器版 Java 虚拟机的编译；
- Asm：汇编器的实现，平台相关部分放到了 CPU 目录中，这个目录主要用来支持解释器，JIT 编译器等生成本地代码；
- C1：JIT 客户版编译器的实现，平台相关的部分放到了 CPU 目录中；
- Ci：JIT 编译器接口的实现，用来支持客户版和服务器版的 JIT 编译器；
- Code：支持运行时生成代码的存储，重定位等；
- Compiler：目录主要是实现一个编译的线程，管理编译队列等；
- Gc_implementation：这个目录主要是 2 种垃圾回收器的实现；
- Gc_interface：这个目录主要是垃圾回收器的接口实现；
- Interpreter：解释器的实现，平台相关的部分在 CPU 目录中；
- Libadt：定义了几个抽象类，用来支持服务器版 JIT 编译器；
- Memory：实现内存管理的功能；
- Oops：定义 Java 对象；
- Opto：实现服务器版 JIT 编译器；
- Prims：实现了 jvmti, jvmpi 接口功能，提供 Java 虚拟机对外接口；
- Runtime：实现了一个运行时环境，平台相关的部分在 cpu，os_cpu，os 中；
- Service：实现了 Java 虚拟机内存管理，线程管理等管理功能；
- Utilities：实现了一些宏定义，异常处理，调试，跟踪等功能，平台相关的部分在 cpu，os_cpu，os 中；

在移植过程中，主要的工作集中在汇编器，解释器，JIT 编译器，运行时环境这 4 个部分，下面的章节将分述这几个部分移植的要点。

4.3 汇编器的移植

HotSpot 的使用汇编器来生成代码，汇编器主要由 2 个类来实现：Assembler,MacroAssembler，主要用来产生本地代码。产生的本地代码由一个静态类 CodeCache 来管理，移植汇编器主要的工作在于生成指令，关键点在于对分支跳转语句，寻址方式，重定位的移植。

4.3.1 代码的存储

汇编器在 CodeBuffer 上生成代码，它用 CodeBlob 的一个子类 BufferBlob 来存储代码，CodeBlob 装载代码所需要的内存从 CodeHeap 中分配，CodeHeap 中维护了一内存块的链表，方便分配，释放内存块。HotSpot 虚拟机定义了一个全局的静态类 CodeCache 来管理 CodeBlob，即生成的代码，如图 4-2 所示。

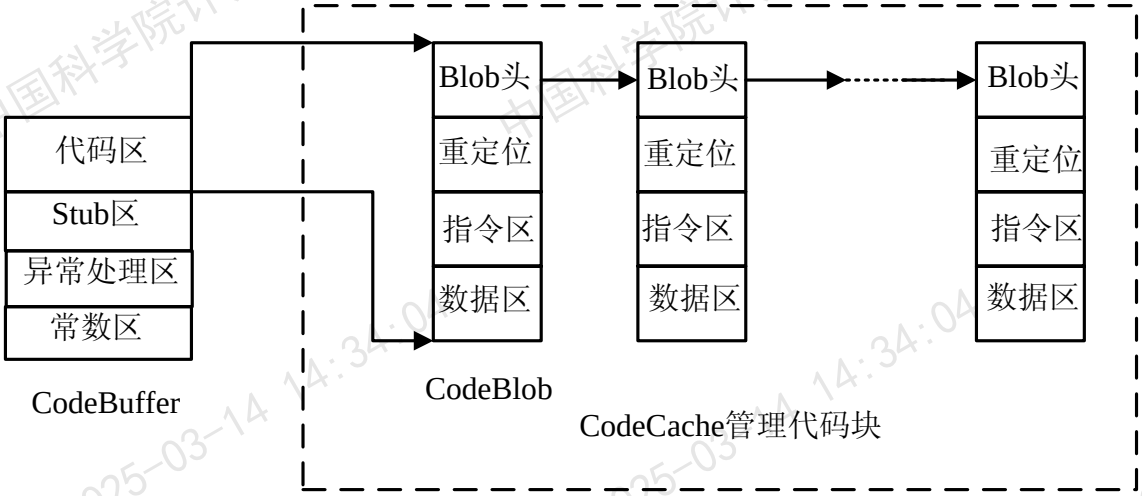


图 4-2 汇编器代码存储结构

CodeBlob 有 7 个子类，用于不同的情况下：

- BufferBlob：用来装载 Java 方法，包括调用本地代码的方法；
- RuntimeStub：用来装载调用运行时环境的方法；
- C2IAdapter：用来装载从编译执行的代码中调用解释执行代码的方法；
- I2CAdapter：用来装载从解释执行的代码中调用编译执行代码的方法；
- DeoptimizationBlob：装载处理从编译执行转向解释执行的代码；
- ExceptionBlob ：用来装载异常处理中处理堆栈展开的代码；
- SafepointBlob ：用来装载位于 Safepoint 状态时处理未知指令异常的代码；

“龙芯”指令集简单，代码长度，格式比较固定，我们在 Java 虚拟机中移植了一个反汇编器，反汇编器可以直接对 CodeBlob 的代码区进行反汇编，便于调试 JIT 编译器。

4.3.2 重定位

重定位是 Java 虚拟机运行过程中一个比较频繁的操作，它主要发生在代码移动和垃圾回收时，它由类 Relocation 实现，与 CodeBuffer，CodeBlob 结合使用。重定位的主要有下面几个功能：

- 在代码移动时修改远程方法调用和远程跳转指令的偏移；
- 垃圾收集器移动对象时，使用这个对象的地方都要更新对象的地址，这个工作由重定位完成；
- 在 JIT 编译器 patch 代码时，提供被修改代码的位置；
- 标志某个代码位置，如方法调用位置，异常发生位置，断点设置位置等；

X86 存在近程方法调用，此时调用的目标和返回的地址也是一个偏移量，而“龙芯”平台的方法调用，方法返回等都使用寄存器寻址方式，这个地址是一个绝对的地址，在 X86 使用近程方法调用，近程跳转语句的地方，在移植中都需要做特殊的处理。当一段运行时代码调用某个函数，而这段运行时代码恰好被移动了，由于 X86 使用的是一个相对地址，不需要任何的重定位，但对于龙芯，由于其使用的是绝对地址，如果不对返回

地址做修改，那么函数将会返回到一个不确定的状态，导致虚拟机崩溃，为了做到这一点，在移植过程中重新定义了一种新的重定位方式处理这种情况。

4.3.3 寻址方式

在 Java 虚拟机中，对内存地址的操作很常见。X86 结构用了好几种寻址方式：直接寻址方式，间接寻址方式，存储器寻址方式等，而龙芯上只有一种寄存器加偏移地址的寻址方式。

对于直接寻址方式，如果立即数大于 16 位，先把立即数装载到某通用寄存器，然后采用该通用寄存器+0 偏移的方式寻址，如果立即数小于等于 16 位，采用 ZERO 寄存器 + 该立即数的方式寻址，此时需要注意的是一些有符号数的极限值，比如说 short 的最小值，由于符号扩展，并不能作为 16 位立即数处理。

对于间接寻址方式，和龙芯的寄存器加偏移的寻址方式一样，不需要做任何处理。

对于存储器寻址方式，即基址+变址*比例因子+偏移的方式，需要增加额外的代码，计算变址*比例因子的值，然后把该值存入基址所在的寄存器中，转化为寄存器加偏移的方式。

4.3.4 分支转移语句

在 X86 中分支转移一般由比较跳转完成，比较主要完成设置符号位，有时候这种设置符号位的指令从表达式上看并不明显，跳转指令根据符号位决定是否跳转。而龙芯中的跳转仅由一条语句完成，在移植过程中需要找出比较的对象，改写成合适的形式。

4.3.5 指令生成

指令生成是汇编器最主要的功能，在移植过程中，参照 X86 版本，用龙芯指令集替换对应的 X86 指令集，完成了这部分的移植。

4.4 解释器的移植

HotSpot 解释器的结构如图 4-3 所示。字节码执行引擎主要完成取字节码，验证字节码，执行字节码的功能；在虚拟机启动时，会生成一个字节码对应的本地代码表，执行字节码时，假如是普通的字节码，则直接从这个表中取对应的本地代码执行；如果是函数调用的字节码，则进入函数调用入口，其中包括 Java 函数入口，本地函数入口等；一个方法的字节码“执行”完后，进入方法返回入口；如果在执行过程中，检测到某些字节码组合可以优化，则对这些字节码进行优化；如果在执行过程中，遇到了异常，则进入异常处理；编译执行接口在方法解释执行过程中完成统计方法执行，判断是否转为编译执行，满足条件则转为编译执行等功能。

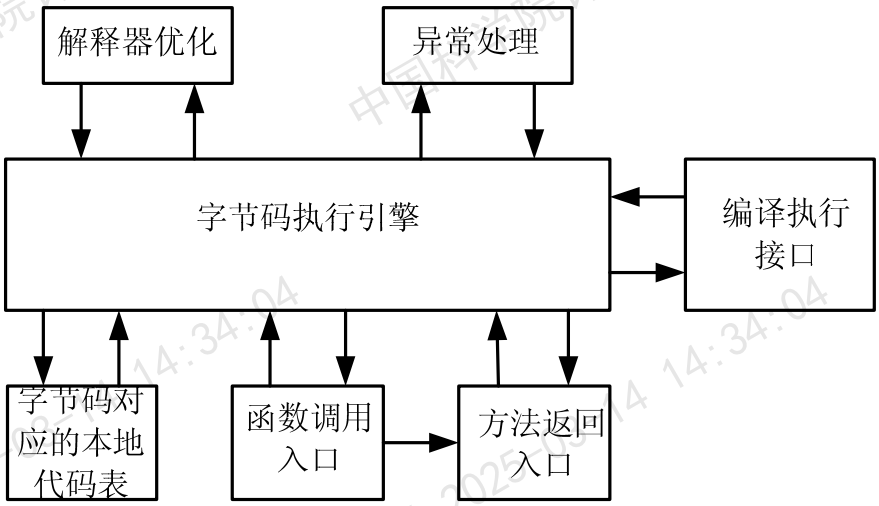


图 4-3 解释器的结构

这 7 个部分中，字节码“执行引擎”和解释器优化这 2 个模块式平台无关的，剩下的 5 个模块都需要在移植中做出修改：

字节码对应的本地代码表是解释器移植中最主要的模块，它由 TemplateTable 类实现，移植的工作主要是根据字节码功能产生本地代码；

函数调用入口是移植中最关键的模块，需要为函数调用设计帧的结构，同时对于调用本地函数需要严格遵循“龙芯”的 ABI 协议，比如说传参数的方式，由于调用本地函数时，第一个参数总是一个指针，所以移植中，直接把前 4 个参数放到 A0，A1，A2，A3 寄存器中，其它参数放到堆栈中，堆栈中预留出 16 个字节的空间，参数在堆栈上的存放注意了存放顺序和对齐；

方法返回的入口：移植中主要做的就是恢复堆栈，同时取下一条字节码；

异常处理：移植中主要作的是给异常处理方法传递必要的参数，并跳转到异常处理方法；

编译执行接口：移植中主要做的是判断是否达到编译执行的条件，如果达到了，通知编译线程开始编译，编译完成后转为编译执行；

总的来说，1.6 版虚拟机的解释器和 1.5 版相比，在平台相关部分变化不大，所以在移植中很多地方可以参考 1.5 版移植的成果，在本章便不做过多论述。

4.5 JIT 编译器的移植

JIT 编译器的移植是 1.6 版 Java 虚拟机移植过程中最难的地方，也是移植工作最集中的地方，新版本的 Java 虚拟机对这部分所作的修改比较多。JIT 编译器编译的过程如图 4-4 所示。 JIT 编译器首先通过扫描字节码，把字节码转换成 HIR 中间语言表示，然后 JIT 编译器对 HIR 中间语言表示进行优化，主要有方法内联，公共子表达式消除，空指针检查消除，条件语句消除等，这一部分一般可以认为是 JIT 编译器的前端，它是平台无关的。接下来，由 HIR 中间语言表示转换为 LIR 中间语言表示，LIR 是一种非常接

近底层平台相关中间语言，寄存器的分配就在这个过程进行，转换为 LIR 中间语言表示后，接下来把这些中间语言表示转换成本地代码，这个过程非常的直接，同时在这个过程中，记录了一些支持动态生成代码运行的数据，这些数据一般称为“Meta Data”，这部分数据主要用来支持从编译执行转向解释执行，记录对象的位置，支持垃圾回收，然后就是支持异常处理。从 LIR 转化为本地代码的过程，一般称为 JIT 编译器的后端，这部分是平台相关的，移植中主要修改的就是这一部分。

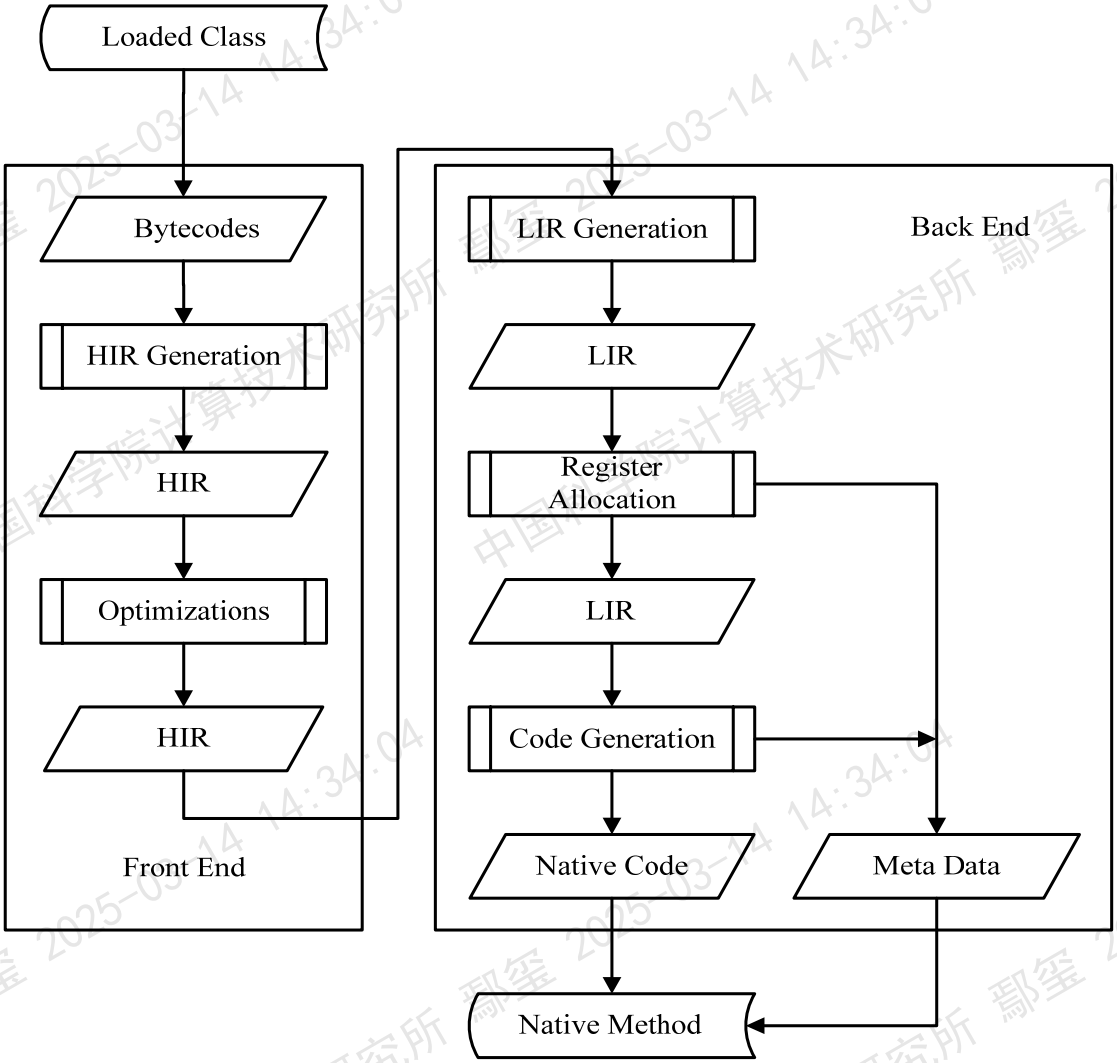


图 4-4 JIT 编译器的工作过程

与 1.5 版的 Java 虚拟机相比，1.6 版 Java 虚拟机变化最大的就是寄存器分配部分，1.6 版采用了线形分配寄存器算法，用来代替 1.5 版中的按需分配，这一节主要论述寄存器分配部分的移植。

4.5.1 线形扫描寄存器分配算法的优势

寄存器分配是影响编译器性能的最终要的因素之一，特别对于 RISC 结构的处理器而言尤为重要。寄存器分配完成的是将程序中的变量用实际的物理寄存器代替的功能，由于寄存器是非常有限的资源，是否能选择较合适的变量放到寄存器中，选择算法的复

杂性是否合理，是衡量寄存器分配算法的 2 个主要因素。

现在的寄存器分配算法主要是 2 种类型：

- 局部寄存器分配算法：顾名思义，这种算法是在当前方法的一个局部范围进行寄存器的分配，这个局部范围一般是循环的最底层。1.5 的客户版 Java 虚拟机使用的是一种按需分配的局部寄存器分配算法：这种分配算法非常的直接，当一条程序语句需要寄存器时，按需分配一个寄存器， 当语句结束时，释放寄存器。寄存器的分配局限在一条语句内，且没有利用寄存器来缓冲局部变量，这个分配过程的开销非常小，时间复杂度为 O(1)，但是生成的代码中，有很多寄存器闲置，处理器持续的访存，执行起来非常的低效。如果利用这些闲置的寄存器来缓冲经常使用的局部变量，可以减少很多访存的操作，从而大幅提高生成代码的执行性能。
- 全局寄存器分配算法：这类算法尝试在整个方法范围内，或者一组方法类，分配寄存器，这类算法能够达到一个比较好的分配结果，但算法的开销很大。1.5 的服务器版Java虚拟机使用的是图染色全局寄存器分配算法，这个算法对所有可以存在寄存器中的变量做了统计，可以选择最合适的变量放到寄存器中。这种算法在现代编译器中被广泛的使用。虽然这个算法经过了多次优化，但现在算法的开销仍然有 $O(n^2)$,n是虚拟寄存器的个数，用在注重峰值的服务器版Java虚拟机中可以接受，但对桌面应用不适合。

对于 JIT 编译器而言，特别是桌面应用的客户版，不仅需要关注寄存器分配的质量，更要关注寄存器分配算法的开销，必须要要在这 2 个中间找一个比较好的折衷。

线形扫描寄存器分配算法线形扫描算法首先线形的排列一个方法中的所有指令，这这次排列中忽略条件转移和循环，然后计算每一个虚拟寄存器的生存周期，生存周期指的是变量第一次定义到最后一次使用这段时期；然后分析程序的条件转移很循环等，最后根据虚拟寄存器的生存周期等信息来分配物理寄存器。算法的时间复杂度为O(n)，n是虚拟寄存器的个数。与图染色寄存器分配算法$O(n^2)$的复杂度相比，线形扫描寄存器分配算法的开销要小很多。同时，与按需分配的算法相比，线形扫描寄存器分配算法在整个方法内分配，能够使寄存器得到充分的利用，能够用多余的寄存器来缓冲局部变量，生成代码的质量将更加优秀，执行效率更高[15]。

终上所述，线形扫描寄存器分配算法在得到更好的代码质量同时，算法时间开销增加小，能够解决龙芯上 JIT 客户版编译器代码生成质量低下的问题，是移植到新版本的 Java 虚拟机的主要动力。

4.5.2 线形扫描寄存器分配算法实现

整个寄存器分配算法的实现需要的类的基本结构如图 4-5 所示。

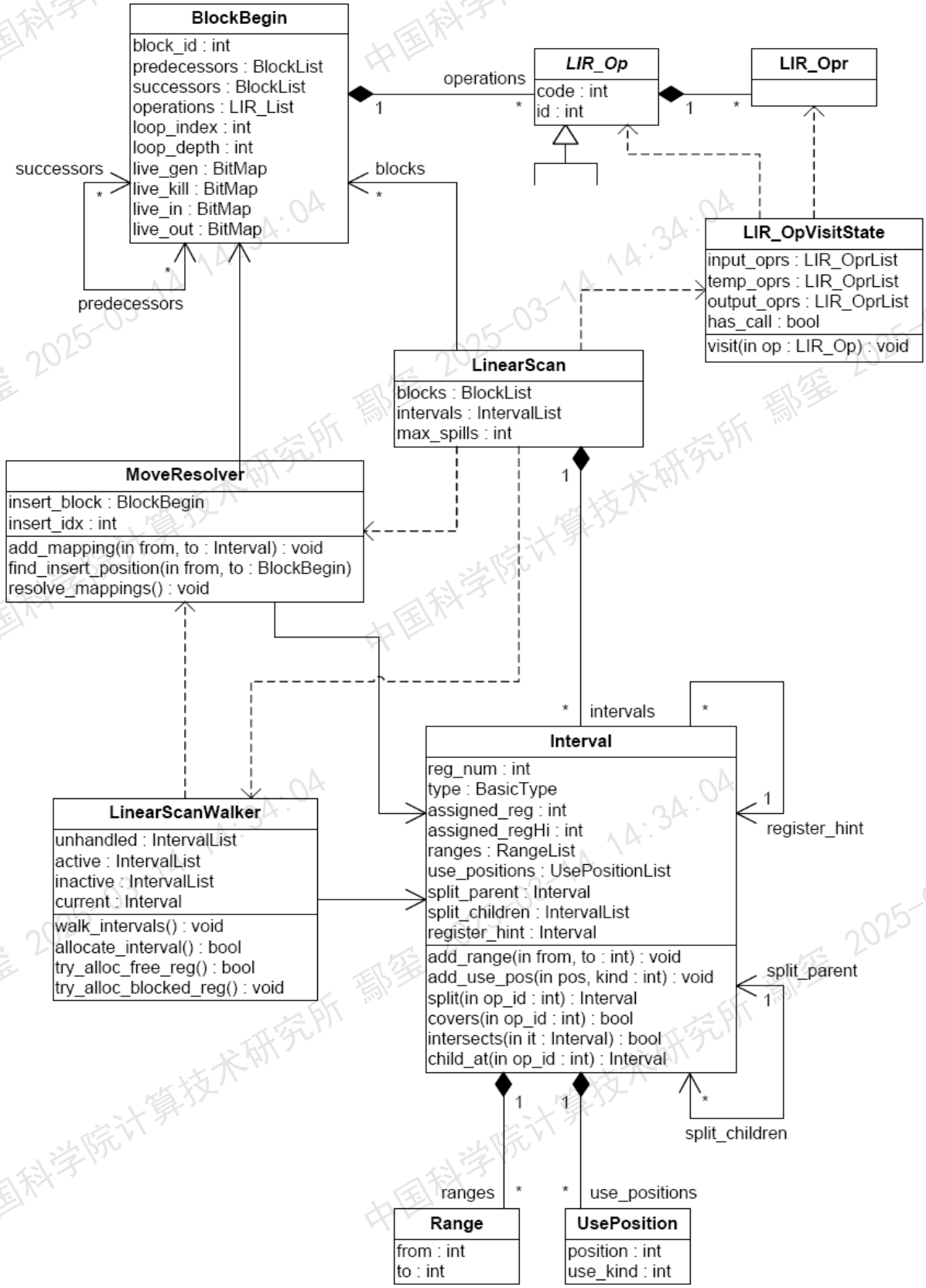


图 4-5 线形寄存器算法实现相关类之间的关系

BlockBegin 类是表示代码块开始的第一条指令；

LIR_Op 类用来表示 LIR 中间代码的指令；

LIR_Opr 类用来表示 LIR 中间代码的操作数；

LIR_OpVisitState 类用来访问中间指令的所有操作数；

Interval 类用来统计计算，统计操作数生存周期，位置等信息，

MoveResolver 类用来处理在 LIR 中间代码中插入 Move 指令，带来的寄存器读写冲突，解决的办法是用来重新排列 Move 指令；

LinearScan 是整个线形扫描寄存器分配算法的核心类；

LinearScanWalker 类是实际进行寄存器分配的类；

线形扫描寄存器分配算法流程的伪码描述如下所示：

```
LINEAR_SCAN
  // order blocks and operations (including loop detection)
  COMPUTE_BLOCK_ORDER
  NUMBER_OPERATIONS
  // create intervals with live ranges
  COMPUTE_LOCAL_LIVE_SETS
  COMPUTE_GLOBAL_LIVE_SETS
  BUILD_INTERVALS
  // allocate registers
  WALK_INTERVALS
  RESOLVE_DATA_FLOW
  // replace virtual registers with physical registers
  ASSIGN_REG_NUM
```

线形扫描算法大致可以分 4 部分完成：线形排列中间代码块，计算操作数的活动周期，寄存器分配，更新 LIR 中间代码等，本节将分述这 4 部分。

(1) 线形排列代码块

线形扫描算法需要首先生成一个代码块的线形排列，寄存器分配的质量和分配的速度与这个线形排列的顺序相关。在实现中，分成 3 步来实现：

第一步：检测方法包含的代码块中的是否存在循环以及循环的层次等，将这些信息记录到一个数据结构中；

第二步：根据循环检测的结构和代码块本来的逻辑顺序确认代码块的排列顺序，代码块排列时按照如下原则：

- 代码块本来的逻辑顺序；
- 判断代码块是否被包含在一个循环当中，有几层循环，循环层次越深，在排列顺序时会尽量靠前；
- 判断代码块是否包含异常处理，如果有异常处理会尽量靠后；

第三步： 在每个代码块内顺序排列中间指令，指令都排列在偶数位置，奇数位置便

于在寄存器不够时插入 spill 操作。

(2) 计算操作数的活动周期

线形扫描寄存器分配算法中，操作数的生存周期和位置是寄存器分配的依据，这个计算的分 3 步完成：

第一步：计算中间代码块内操作数的生存周期，形成局部活动集合。这一步所做的工作主要是：对每个代码块，填充如图所示 BlockBegin 中定义的这 2 个数据结构：

- live_gen：在本代码块内使用，但在之前的代码块中定义的操作数的集合；
- live_kill： 在本代码块中定义的操作数集合，这个定义的过程会改写之前定义的值；

第二步：使用标准的回朔数据流分析算法，计算每个代码块开始和结束处的操作数生存情况，形成全局活动集合。这一步完成的主要工作是： 对每个代码块，填充在 BlockBegin 中定义的这 2 个数据结构：

- Live_out：在本代码块入口处输入的，存活的操作数集合；
- Live_in： 在本代码块出口处输出的，存活的操作数集合；

第三步：利用局部活动集合和全局生存集合精确的计算出每个操作数的生存周期。这个过程主要是填充在 Interval 定义的 2 个数据结构：

- _current：当前操作数的生存周期；
- _use_pos_and_kinds：当前操作数的位置和类型；

计算的过程如下所示：

- 对于 live_out 中每一个操作数，初始化设置该操作数的生存周期是从代码块开始一直到结束，如果该代码块中重定义了这个操作数，则把该操作数的生存周期更新为从重定义地点到代码块结束；
- 从代码块的最后一条操作数开始，往前面遍历所有的操作数，对于输入的操作数，遇到使用的地方，设置该操作数的生存周期为代码块开始到使用地方，这个过程不仅处理虚拟寄存器生存周期，还处理物理寄存器，对于物理寄存器，分配一个固定的 Interval 结构，避免后续的处理；
- 检测操作数的生存周期是否重合，如果重合，则合并操作数的生存周期，记录当前操作数的位置和类型等；

(3)寄存器分配

经过前面的处理后，虚拟机寄存器的信息保存到了 Interval 中，这个阶段处理 Interval 结构，把虚拟寄存器，映射到实际的物理寄存器，这一阶段的工作是寄存器分配算法的核心，主要由类 LinearScanWalker 处理，这个处理过程由 3 步完成：

第一步：设定 Interval 对象的状态，Interval 对象主要有下面 4 个状态：

- unhandleState： 表示还没有处理；
- activeState：表示该操作数处于生存周期，且已经分配了物理寄存器；
- inactiveState：表示该操作数处于生存周期洞中(从操作数上一次使用到重新定义

这段时间)，且已经分配了寄存器；

- handledState：表示操作数被 spill，或者不处于生存周期；

在寄存器分配，首先从 uhandleState 的 Interval 中取一个位置最靠前的对象，得到这个对象在代码块的位置，根据这个位置值更新 activeState,inactiveState 对象的状态，更新的结果，或者变为相反的状态，或者为 handleState。这些 Interval 对象的状态和位置影响下面的寄存器分配；

第二步：尝试分配空闲的寄存器。分配的原则是：

- 假如有空闲的寄存器，则直接分配；
- 如果有物理寄存器属于 inactiveState 的 Interval，且当前 Interval 的地址位于处于 inactiveState 状态的 Interval 的 lifetime hole 中（操作数从上次使用到重新定义的范围），可以先将该寄存器分配给当前 Interval，到 inactiveState 的 Interval 重定义时收回，此时，如果当前 Interval 仍然生存，再创建一个新的 Interval，设置为 unhandleState 状态；
- 假如所有的物理寄存器都是都属于 activeStatie 的 Interval，则没有寄存器可以分配，进入下一步的处理；

第三步：如果没有空闲的寄存器，尝试 spill 部分寄存器（把部分寄存器的值保存到内存），然后分配空闲出来的寄存器，spill 的处理有几个原则：

- 因为循环内语句执行次数多，不能在一个循环内插入 move 操作进行 spill；
- 尽量选择代码块边界处插入 move 操作的操作数进行 spill；
- 选取奇数位置插入 move 操作进行 spill；
- Spill 操作后空闲的寄存器便可以用来分配了。

第四步：如果当前 Interval 已分配物理寄存器，则设定其状态为 activeState。

4.5.4 更新 LIR 中间代码

寄存器分配完成后，分配的结果保存在 Interval 对象中，需要把这些结果更新到 LIR 代码，同时，在寄存器分配过程中，可能在中间代码中插入了一些 move 指令做寄存器的 spill，由于分支转移指令的存在，需要合理的安排这些 move 指令的顺序，避免对寄存器复写操作。这一步主要完成这 2 个工作。由于 LIR 中间代码中，move 操作有 50% 以上，这一步还做了一些优化，比如公共 move 操作合并等，来减少这个操作。

4.6 运行时环境的移植

运行时环境用来支持 Java 虚拟机的运行，它的功能包括线程支持，同步，垃圾回收支持，定义各种栈帧格式，支持方法调用，支持解释器执行，编译执行以及 2 种执行转换时堆栈处理，异常处理，“运行时数据”统计，支持调试等。在移植中，最关键的工作是一系列栈帧的移植，与前版本相比，同步机制变化较大，引进了 Biased lock，本节将

分述 Java 虚拟机中的栈帧和同步机制。

4.6.1 Java 虚拟机的栈帧

每个方法（无论是 Java 方法还是本地方法）的调用都会创建一个帧，这个帧根据方法的类型有不同的结构，这个结构是平台相关的，需要在移植过程中根据 MIPS ABI 文档设计。由 FP（帧指针），SP（栈指针），PC（程序指针）这 3 个值可以唯一的确定一个帧，这 3 个变量和一系列方法构成了一个类：Frame。Frame 可以分成很多种，如图 4-6 所示。

- 如果 Frame 在 Java 方法中建立，则被称为 Java Frame，其中又可以分为 Interpreter Frame 和 Compiled Frame；
- 如果 Frame 在本地方法中建立，则为称为 Native Frame；
- 如果 Frame 是一些辅助调用的方法所建，称为 Glue Frame，其中又可以分为 Runtime Frame，I2c Frame，C2i Frame 等，这些 Frame 所决定的帧结构简单，只是做为方法调用的一个中转；
- 如果 Frame 是为了从虚拟机中调用 Java 方法所建，称为 Entry Frame；

Frame 在源代码级别的表示是 Vframe，Vframe 类包含了 Frame 类，Frame 所属的线程，寄存器表（RegisterMap）这 3 个变量和一系列的操作。Vframe 和 Frame 的对应关系如图 5-6 所示，值得注意的是由于 Glue Frame 只是做为函数调用的中转，没必要创建对应的 Vframe。Java 方法从编译执行转为解释执行时（deoptimization），对堆栈的操作是移植的难点，因为 JIT 编译器编译时会对合适的小方法进行内联优化，在 deoptimization 时，一个 Compiled Frame 可能和多个 Interpreter Frame 对应。由于在生成 IR 中间代码时，在方法内联之前记录了方法调用的一些信息，由这些信息可以生成一个 Vframe 数组，将当前的 Compiled Frame 出栈后，根据 Vframe 数组的信息生成一对应的新的 Interpreter Frame（unpack_on_stack），然后把这些新生成的 Interpreter Frame 决定的帧压入堆栈，可以解决这个问题。

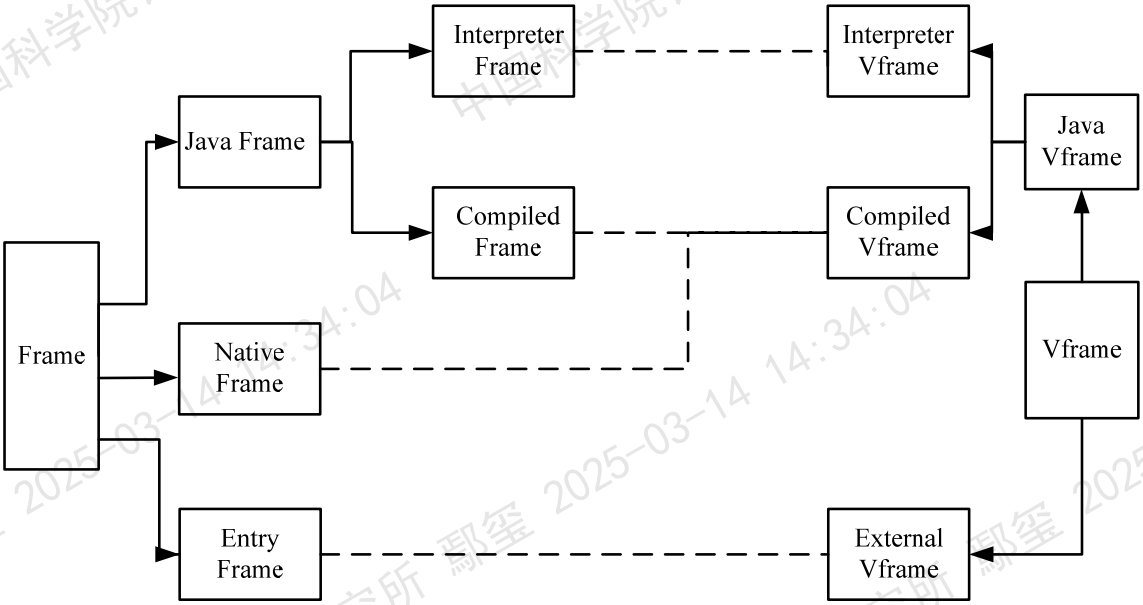


图 4-6 Java 虚拟机中的栈帧

4.6.2Java 虚拟机的同步机制

Java 语言内置了同步的支持，本节讲论述同步在 Java 虚拟机中的实现，即 Java 虚拟机对 monitorenter，monitorexit 这 2 个字节码的处理。所有的 Java 对象头部第一个字（Mark Word）记录了该对象关于同步，垃圾回收等的信息，如表 4-1 所示。

表 4-1 Java 对象的 MarkWord 结构

Bitfield				Tag bits	State
Hash		Age	0	01	unlocked
Ptr to lock record				00	Lightweight locked
Ptr to heavyweight monitor				10	inflated
				11	Marked for GC
Thread ID	Epoch	Age	1	01	biasable

Java虚拟机的同步机制有轻锁（lightweight monitor），重锁(heavyweight monitor)之分，由于“重锁”需要调用操作系统线程的上锁，解锁功能，开销非常大，所以，Java虚拟机优先使用轻锁，这主要是使用CAS(compared and swap)原子操作来改写MarkWord。在轻锁失败时，即在竞争条件下，才使用重锁，在Linux系统中，是使用pthead库的mutex机制。在 1.6 版Java虚拟机中，引进了一种新的轻锁：biased lock，如表 5-1 所示，使用了Mark Word定义了一个新的bit位来区分普通“轻锁”和biased lock。由于Java程序运行中大量的Monitor不仅是非竞争的，而且在大多数情况下整个生存周期只有单个线程访问[18, 19, 20]，基于这点，上锁的时候用普通的测试分支（test and branch）来代替CAS原子操作，这个过程开销非常的小，特别是在多CPU模式下，可以大幅度的节约原子操作需要的时间，但假如有 2 个线程同时使用了Monitor，需要从biased lock转为普通的lock,这个

过程将带来额外的开销，总的来讲，节约的时间远多于这个额外的开销[21, 22]。出于效率的考虑，“轻锁”的实现均使用运行时代码完成，是需要移植的部分，重锁的实现直接调用pthread库的系统调用，移植中只需要考虑判断是否采用重锁和怎样调用重锁等。

4.7 本章总结

为了改进动态生成代码的性能，同时也为了支持最新的 Java 特性，开始了对 1.6 版虚拟机的移植。本章对移植过程中平台相关，且与前版本 Java 虚拟机有差异的地方进行了论述，这些地方包括汇编器，解释器，JIT 编译器，运行时环境。其中，重点论述了差异最大的 JIT 客户版编译器的寄存器分配算法的分析与移植。1.6 版 Java 虚拟机引进的线型扫描寄存器分配算法，由于这种寄存器分配算法能够从整个函数体内分配寄存器，同时可以用多于的寄存器缓冲局部变量等，能够充分的使用“龙芯”的寄存器资源，减少访存操作，于是，可以在一定程度上解决前一章所述的生成代码质量低下的问题。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢銮 2025-03-14 14:34:04

第五章 结论和展望

5.1 本文工作总结

本文首先论述了 Java 虚拟机的体系结构，在此基础上论述了对 Java 虚拟机的调试，通过对 Java 虚拟机的调试分析，大大提高了 Java 虚拟机的稳定性和可用性，现在桌面应用的 Java 程序基本上可以“龙芯”平台上执行，包括永中 Office 这样的大型 Java 软件；然后，本文对 Java 虚拟机性能的进行了分析，根据分析的结果，提出了解决对应的性能瓶颈的方案，通过将代码移植到新的 GCC 编译器，改写 Cache 操作，消除对不对齐地址访问等，调整内部执行参数，用多余寄存器缓冲运行时代码的一些关键变量等，Java 虚拟机性能用 JVM98 测试平均执行时间减少了 60%左右，即性能提高了 160%左右。为了进一步解决 JIT 编译器生成代码性能低下的问题，开始了对 JDK1.6 的分析和移植，首先分析了移植 JDK1.6 的必要性，然后论述了移植过程中的一些典型问题，包括解释器的移植，JIT 编译器的移植，运行时环境的移植等。

综观本文的工作，最关键的部分在于 Java 虚拟机性能分析以及优化这一部分，在这一部分，本文主要采用了利用 Java 虚拟机内部自带的执行统计选项、Oprofile 进行性能分析，前者主要用来从整体上定位 Java 虚拟机性能不佳的部分，缩小分析范围，后者主要用来定位性能瓶颈的细节，但目前的分析方法只适合于性能瓶颈在本地函数的情况，对于运行时生成的函数，利用 Hprof 可以分析其性能，但对于运行时环境，目前还缺乏比较有效的手段，在优化中，这一部分主要是借鉴别人的分析结果。经过本文的工作，基本上解决了 Java 虚拟机本地函数方面的瓶颈，目前，以 JVM98 为工作集，Java 虚拟机运行 Java 程序时只有 30%左右的时间花费在本地函数的执行上，所以，下一部分性能优化的重点在运行时环境和 JIT 编译器上。

在移植和优化 Java 虚拟机的过程中，缺乏丰富的调试手段是最大的困难，由于本身 Java 执行的特殊性：动态生成代码和本地代码混合执行，传统的调试工具，比如说 GDB，在很多时候不能胜任，虽然 Java 虚拟机的设计考虑了可测性，提供了一些调试手段，但这些调试手段不足以应付完全移植到另外一种体系结构的调试。调试手段的缺乏带来的后果就是当发现问题时，调试的效率不高，所以，研究 Java 虚拟机的调试，发掘更好的调试手段对龙芯上 Java 虚拟机的调试具有重大的意义。

5.2 下一步的工作

现在龙芯上 Java 环境和 X86 主流平台相比还有较大差距，所以，对 Java 虚拟机的优化任重而道远。现在，Java 虚拟机的性能还是有潜力可发掘的：

- 把 Java 虚拟机移植到“龙芯”这种 RISC 结构时，和 X86 相比，有诸多的不便，比如说：目前的移植在函数调用，返回均使用寄存器绝对寻址的方法，对比 X86 在局部范围内可以使用偏移地址来调用函数，如果调用者代码移动了，需要额外的重定位操作来保证返回地址的正确性。且寄存器寻址的方式不利于 CPU 的分支预测，不能充分发挥 CPU 的性能。这个问题可以通过改写运行时生成方法的调用，返回方式解决。比如说在调用地址或返回地址与当前地址相差在 64k 以内时使用 Branch 的方式调用方法，从方法中返回；
- Java 面向对象的语言特性，方法调用非常频繁，由于“龙芯”平台的寄存器非常多，在函数调用和返回时保存，恢复寄存器的开销比 X86 结构大很多。通过尽可能的内联可以减少方法的调用，从而减少这个开销，但这种方案需要大内存的支持；
- “龙芯”也存在一定的平台优势，比如说通用寄存器数目众多，可以尽可能的利用寄存器存储一些运行时环境的一些关键变量，减少访存操作，这也是今后需要改进的地方。

现在对 Java 虚拟机的工作主要集中在客户版 Java 虚拟机上面，这主要是桌面应用，在这个方面要作的工作是继续对 1.6 版 Java 虚拟机进行移植和优化，为了针对服务应用，下一步还计划展开对服务器版 Java 虚拟机的移植和优化。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

参考文献

[1] The Java HotSpot™ Virtual Machine Technical White Paper,Sun Microsystems, Inc.2001

[2] The evolution of a high-performing Java virtual machine , by W. Gu,N. A. Burns,M. T. Collins,W. Y. P. Wong. IBM SYSTEMS JOURNAL, VOL 39, NO 1, 2000

[3] A unifying approach to performance analysis in the Java environment. by W. P. Alexander,R. F. Berry,F. E. Levine,R. J. Urquhart IBM SYSTEMS JOURNAL, VOL 39, NO 1, 2000

[4] The Jalapeno virtual machine. by B. Alpern,V. Litvinov,C. R. Attanasio, M. F. Mergen,J. J. Barton,T. Ngo,M. G. Burke,J. R. Russell,P. Cheng,V. Sarkar,J.-D. Choi,M. J. Serrano,A. Cocchi,J. C. Shepherd,S. J. Fink,S. E. Smith,D. Grove,V. C. Sreedhar,M. Hind,H. Srinivasan,S. F. Hummel,J. Whaley,D. Lieber IBM SYSTEMS JOURNAL, VOL 39, NO 1, 2000

[5] Overview of the IBM Java Just-in-Time Compiler. by T. Suganuma,T. Ogasawara,M. TakeuchiT. Yasue,M. Kawahito,K. Ishizaki,H. Komatsu,T. Nakatani IBM SYSTEMS JOURNAL, VOL 39, NO 1, 2000

[6] Application of the HotSwap Technology to Advanced Profiling. M. Dmitriev Sun Microsystems Laboratories, 901 San Antonio Road, Palo Alto, CA 94303, USA June 5, 2002

[7] JavaTM 2 Platform, Standard Edition 5.0 Trouble-Shooting and Diagnostic Guide.Copyright (c) 2006 Sun Microsystems, Inc., 4150 Network Circle, Santa Clara, California 95054, U.S.A. All rights reserved.

[8] The JavaTM Virtual Machine Specification Second Edition Tim Lindholm Frank Yellin.Sun Microsystems, Inc.

[9] Inside The Java Virtual Machine, 2nd Edition,Bill Venners,ISBN : 0-07-135093-4 ,2000

[10] SYSTEM V APPLICATION BINARY INTERFACE MIPS® RISC Processor Supplement 3rd Edition

[11] Characterizing the SPEC JVM98 Benchmarks On The Java Virtual Machine. Kyle R. BowersDavid Kaeli. Northeastern University Computer Architecture Research Group Department of Electrical and Computer Engineering.Boston, Massachusetts 02115

[12] An Execution Profiler for Modular Programs. S.L.Graham, P.B.Kessler,and

M.K.McKusick. software--practice and Experience 13.No,8,671-685(August 1983)

[13] The Java HotSpot Performance Engine Architecture, White Paper,Sun Microsystems,inc,Palo,Alto,CA(April 1999)

[14] kaffe虚拟机相关资料http://www.kaffe.org/doc/kaffe/

[15] Linear Scan Register Allocation for the Java HotSpot Client Compiler. Institute for System Software Johannes Kepler University Linz Linz, August 2004

[16] Matthias Jacob,Keith Randall,CrossArchitectural Performance Portability of a Java Virtual Machine Implementation,Proceedings of the 2nd JavaTM Virtual Machine Research and Technology Symposium (JVM '02) San Francisco, California, USA August 1-2, 2002

[17] Memory Management in the Java HotSpot Virtual Machine Sun Microsystems April 2006

[18] Bacon, D. F. and Fink, S. Method and apparatus to provide concurrency control over objects without atomic operations on nonshared objects. U.S. Patent Number 6,772,153 issued August 3, 2004.Assignee: International Business Machines Corporation.

[19] Gomes, B. A., Bak, L., and Stoutamire, D. P Method and apparatus for speculatively locking objects in an object-based system. U.S. Patent Number 6,487,652, issued November 26, 2002. Assignee: Sun Microsystems, Inc.

[20] Kawachiya, K., Koseki, A., and Onodera, T. Lock reservation: Java locks can mostly do without atomic operations. In proceedings of OOPSLA '02, November 2002, pp. 130–141.

[21] Onodera, T., Kawachiya, K., and Koseki, K. Lock reservation for Java reconsidered. In proceedings of ECOOP '04, June 2004, pp. 559-583.

[22] Kenneth Russell,David Detlefs,Eliminating Synchronization-Related Atomic Operations with Biased Locking and Bulk Rebiasing,Sun Microsystems,OOPSLA'06 October 22–26, 2006,

[23] Oprofile相关资料http://oprofile.sourceforge.net/docs/

[24] Diego Novillo,Tree SSA A New Optimization Infrastructure for GCC,Red Hat Canada, Ltd.GCC Developers Summit 2003 182~193

[25]Danny Coward, Mark Reinhold, What's New in Java SE 6,http://java.sun.com/developer/technicalArticles/J2SE/Desktop/javase6/

致 谢

很幸运能够加入计算所，使得我能够在这 3 年遇到了这么多好的老师和同学，度过充实的三年。

首先感谢张福新，吴少刚，胡明昌 3 位老师的指导和关怀，不仅为我提供了良好的环境和资源，还在工作给了我关键的指导，在我遇到困难时为我指明了正确的解决方向。

感谢章军老师带我进入数字集成电路设计的领域，在章军老师手下做项目的一年是我成长很快的一年，感谢苏州中科在这段时间提供的对我生活和工作上的支持。

感谢肖永田，章世华，张美林，管至宾，孙丰，刘虎等一起参加 SoC 项目的同学，同事，能够和你们在一起通过良好协作，做完一件事情，是很有缘的。

感谢彭亮锦，晏华，张斌，刘军良等同事，和你们一起共事，让我对公司的前途充满信心。

感谢华高，海东，波波，小凡，晓云，陈军，刘云，张标等一群死党，我们一起度过了快乐的研一生活。

感谢挚友小雨在我低沉的时候给我关心和鼓励，帮我度过一段艰难的岁月，感谢老妹一直能坚持听我絮絮叨叨，给我有用的建议。

谨以此文献给我的父母，感谢你们一直以来，承担着家庭的压力，生我养我教育我 20 多年。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04

作者简历

姓名：陈 杰　　性别：男　　出生日期：1981.10.19　　籍贯：湖南南县

2004.9 -- 2007.7　　中科院计算所系统结构室硕士研究生
2000.9 -- 2004.7　　华中科技大学计算机科学与技术学院本科生

【攻读硕士学位期间发表的论文】

- 陈杰，章军 ，“一种集成龙芯 IP 核的 SoC 体系结构”，*计算机工程与应用*（已录用）

【攻读硕士学位期间参与的科研项目】

- 05.07-06.07　“基于龙芯 1 号的税控 SoC 芯片设计”（江苏省科技成果转化专项资金项目）
- 06.07-今　“低成本先进计算机系统”（中科龙梦公司承担的国家 863 重点项目）

中国科学院计算技术研究所 鄢玺 2025-03-14 14:34:04